Apunte_Maestro_Parcial2_TDI

Apunte Maestro — Parcial 2 de Teoría de la Información

*Refinado a partir de: los apuntes de cátedra de Fabricio Rubio (Unidades 7, 8, 9 y 10), las presentaciones de la cátedra 2026 (**Tema 1 Introducción a la Criptografía, Tema 2 Sistemas clásicos, Tema 3 Cifrado simétrico en bloques, Tema 4 Cifradores de flujo, Tema 5 Cifrado asimétrico exponencial**), el **Práctico de Criptografía 2026** (Área Temática 2) y **todos los parciales 2 disponibles** (2010, 2012 T1/T2, 2013 T1/T2, 2020 con resolución aprobada de una alumna, 17-11-2025 Tema 1 con resolución de una alumna, y las «Preguntas teóricas parcial 2»). Todos los cálculos numéricos (polinomios, matrices, distancias mínimas, descifrados, RSA, LFSR) fueron **recalculados por computadora**, no a mano; donde una resolución de alumno no coincide con el cálculo, está marcado (sección 12).*

Cómo usar este documento

El Parcial 2 es **Codificación de canal (control de error) + Criptografía**, y según el año suma una pregunta de **imágenes** y otra de **teóricas**. Se lee mejor en este orden:

1. **Mapa del examen (sección 0)**: qué entra, cuántos puntos vale cada bloque y —lo más útil— que **los códigos de los parciales se repiten** (tres códigos cíclicos explican 7 de los 8 exámenes). Los resultados ya están calculados.
2. **Teoría (secciones 1 a 3)**: control de error, criptografía, imágenes. Sirve para las teóricas y para entender por qué sale cada cuenta.
3. **Recetario + ejercicios resueltos (secciones 4 y 5)**: los pasos exactos de cada tipo de ejercicio y los parciales/prácticos resueltos. **Si tenés poco tiempo: leé 0, 4, 5, 8 y 12.**
4. **Chuleta (6), teóricas con respuesta (7), correcciones a resoluciones de alumnos (12).**

> **Convención de notación (clave para no perder puntos).** Los parciales escriben los polinomios con **«mayores potencias a la derecha»**: la cadena de bits `b0 b1 b2 …` es $b_0 + b_1 x + b_2 x^2 + \dots$ (el primer bit es el coeficiente de x^0). Los apuntes de Fabricio, en cambio, a veces escriben `(1,1,0,1)` como $x^3 + x^2 + 1$ (mayor potencia a la izquierda). **Leé siempre el enunciado y escribí la convención que usás.** En este apunte, salvo aviso, los bits se escriben **de menor a mayor potencia** (como los parciales).

---

0. Qué se evalúa en el Parcial 2 (mapa rápido)

0.1 Estructura de los exámenes

Año / tema	Control de error	Criptografía	Otros
2010	Código (12,K) con $h(x) = x^3 + x^2 + x + 1$: capacidad, qué bits controla cada chequeador, decodificar `110011001110`, **dibujar el circuito** de $h(x)$	Vigenère `RACING` + permutación (6,5,4,3,2,1); **entropía de cada etapa**, qué orden de entropía modifica cada técnica, redundancia	Teóricas: ordenar `.exe/.txt/.rar` por entropía, entrelazar 7 códigos, simétrica vs. asimétrica, qué fuente conviene comprimir
2012 T1	(15,K) con $h = x^4 + x + 1$: $g(x)$, capacidades, chequeadores, validar/corregir $R(x) = x^8 + x^4 + x^2$, ráfaga con 20 códigos	Transposición 3×4 + Vigenère `CANCUN`; ¿modifican la entropía de orden 0?	Imágenes (máscaras de rectas), teóricas: $I(A;B)$ de un buen sistema, propiedades de $g(x)$
2012 T2	(14,K) con $h = x^4 + x^2 + x + 1$ (igual que 2013 T1), mismo formato	Vigenère `AZULES` + transposición 4×4; ¿cuál da más seguridad?	Imágenes (suavizado con media), teóricas: base prima, ISBN, **cuadro de Galois / Reed-Solomon**
2013 T1	(14,K), $h = x^4 + x^2 + x + 1$	Vigenère `KERBEROS` + transposición 4×5 clave (2,5,3,1,4); confusión/difusión; entropías	—
2013 T2	(12,K), $h = x^3 + x^2 + x + 1$, matriz G, entrelazar 20; **ISBN-10** `1-122-?3445-5`	Vigenère `HELLMAN` + transposición 3×7; simétrico vs. asimétrico, **cifrado híbrido**; $I(A;B)$	—
2020 (15-12)	(12,3): k, distancia mínima, $g(x)$, G y H, **triple error y síndrome** (3,5 pts); **ISBN-10** `3-540-?3218-9`, base prima (3 pts)	**Afín** $(a, b) = (3, 11)$, $n = 29$ (2 pts)	Imágenes (1,5 pts): vectorial vs. bitmap, **RLE de PCX**
17-11-2025 T1	**Cíclico (14,4)** dado por $G(x)$: capacidad, G y H, validar y corregir, entrelazado ×25; **ISBN-13**; **código rectangular/producto**	**Afín** $(13, 9)$, $n = 40$ con alfabeto de 40 símbolos; ¿qué algoritmo conserva la entropía de orden 0?	Imágenes: **filtro paso bajo** (mediana)
«Preguntas teóricas P2»	Entrelazado, propiedades de $g(x)$, ISBN, tabla Reed-Solomon, G vs. H, capacidad	Entropía de archivos, simétrica vs. asimétrica, híbrido, qué comprimir,	—

Año / tema	Control de error	Criptografía	Otros
	de un código con $d = 10$	$I(A;B)$, base prima, confusión/difusión, entropías por algoritmo	

Lectura de la tabla. Cada parcial tiene un ejercicio de **control de error con un polinomio** (casi siempre cíclico), un ejercicio de **criptografía clásica** (casi siempre «descifrá este criptograma + decí qué entropías cambia cada paso») y teóricas que se repiten años tras año. El 2020 y el 2025 agregan **ISBN** (asignaturas de «código pesado») y **imágenes**.

0.2 Los códigos se repiten (resultados ya calculados y verificados)

Los enunciados dan el **polinomio de chequeo** $h(x)$ (de grado k) y el largo n; el generador es $g(x) = \dfrac{x^n + 1}{h(x)}$ (grado $n - k$). Tres códigos explican casi todos los exámenes:

Código	Aparece en	k	$g(x)$ (bits, x^0 primero)	Pesos de las palabras	d_{min}	Corrige / detecta	Redundancia	Ráfaga con L entrelazados
(12, 3) $h = x^3 + x^2 + x + 1$	2010, 2013 T2, 2020	3	$x^9 + x^8 + x^5 + x^4 + x + 1 \rightarrow$ `1100110011`	6 y 12	**6**	corrige 2, detecta 5	$9/12 = 75\%$	$L \cdot 2$ (con $L = 20$: 40; con $L = 7$: 14)
(14, 4) $h = x^4 + x^2 + x + 1$	2012 T2, 2013 T1, **2025 T1**	4	$x^{10} + x^8 + x^7 + x^3 + x + 1 \rightarrow$ `11010001101`	6, 8 y 14	**6**	corrige 2, detecta 5	$10/14 = 71{,}4\%$	$L \cdot 2$ ($L = 20$: 40; $L = 25$: **50**)
(15, 4) $h = x^4 + x + 1$	2012 T1	4	$x^{11} + x^8 + x^7 + x^5 + x^3 + x^2 + x + 1 \rightarrow$ `111101011001`	todas 8	**8**	corrige **3**, detecta 7	$11/15 = 73{,}3\%$	$L \cdot 3$ ($L = 20$: 60)

Atajo que cierra con los tres: $d_{min} \leq$ (cantidad de términos de $g(x)$), porque $g(x)$ es una palabra del código; en estos casos da igualdad (6, 6 y 8 términos). El profesor lo usa como «la cantidad de coeficientes de $g(x)$ me da d_{min}».

0.3 Qué tengo que saber hacer (checklist)

- [ ] Hallar k, $g(x)$, capacidades, G y H de un cíclico (sección 1.9 y recetario 4.1–4.4).
- [ ] Decir **de qué bits de información depende cada chequeador** (4.3).
- [ ] Validar $R(x)$ por división, hallar el síndrome y **corregir** (4.4).
- [ ] Entrelazado: capacidad de ráfaga (1.12).
- [ ] ISBN-10 (dígito faltante, capacidad) e ISBN-13 (1.4).
- [ ] Código rectangular / producto: $d_{min} = d_1 \cdot d_2$ (1.5).
- [ ] Tabla de **campos de Galois / Reed-Solomon** (1.11).
- [ ] César / afín / Vigenère / transposición: cifrar, descifrar, hallar la clave inversa (2.4, 4.6–4.8).
- [ ] **Qué orden de entropía modifica cada algoritmo** (2.5) y qué fuente conviene comprimir.
- [ ] Simétrico vs. asimétrico, híbrido, firma digital, RSA, Diffie-Hellman, ElGamal (2.7–2.9).
- [ ] LFSR: dibujar, secuencia, período, polinomio primitivo (2.6).
- [ ] RSA a mano (clave pública válida, d, cifrar/descifrar, firmar, Fermat) (4.9–4.10).
- [ ] Imágenes: filtros, máscaras, RLE (sección 3).

---

1. Control de error: Unidades 8 y 9

1.1 Errores y redundancia

El canal introduce ruido que cambia bits. Hay dos tipos:

- **Errores aleatorios (independientes):** la causa de un error no se relaciona con la de otro; se asocian al ruido blanco. Probabilidad pareja en cualquier posición.
- **Errores en ráfaga:** un grupo de bits **consecutivos** afectado por una causa común (un rayón en un CD, un desvanecimiento). **Longitud de ráfaga** B = bits desde el primer error hasta el último.

El control de error **agrega bits de redundancia**. Un código **detector** avisa que hubo error (se pide retransmisión o se descarta); uno **corrector** además lo repara, pero necesita bastante más redundancia. La corrección se usa cuando retransmitir es inviable o ineficiente (TV digital, tiempo real, satélite, discos). En la materia solo se ven códigos **lineales**.

Para proteger k bits de información se transmiten n bits (palabra válida o palabra código), con $n-k$ bits de redundancia (**chequeadores**). Se escribe (n,k) o (n,k,d_{min}).

- **Redundancia del código** $= \dfrac{n-k}{n} \cdot 100\%$ (cuanto menor, mejor).
- **Tasa** $= k/n$.

1.2 Distancia de Hamming, distancia mínima y capacidad

- **Distancia de Hamming** $d(a,b)$: cantidad de posiciones en que dos palabras de igual largo difieren. $d(0111,0101)=1$, $d(1000,1111)=3$, $d(0110,1001)=4$.
- **Distancia mínima** del código: $d_{min} = \min\{d(a,b) \mid a,b \in C,\ a \neq b\}$.
- **Visión geométrica:** cada palabra válida es el centro de una esfera; d_{min} es la separación entre centros.

Capacidad según d_{min} (fórmulas que hay que saber de memoria):

Uso	Capacidad
Detector	detecta hasta $d_{min}-1$ errores
Corrector	corrige hasta $t = \left\lfloor \dfrac{d_{min}-1}{2} \right\rfloor$ errores, o sea $d_{min} \geq 2t+1$
Corrector + detector	corrige t y además **detecta hasta** $d_{min}-1-t$ (el «3.er error» de $d=6$)

d_{min}	Como detector	Como corrector: corrige	…y además detecta
2	1	0	1
3	2	1	1
4	3	1	2
5	4	2	2
6	5	2	3
7	6	3	3
8	7	3	4
10	9	**4**	**5**

Ejemplos que el profesor usa: $d=6$ detecta 5 o corrige 2 y detecta un tercero; $d=10$ «corrige 4, detecta 5» (como detector, 9); $d=128$: corrige 63 y detecta 64 (como detector, 127).

Qué pasa si hay más errores que la capacidad (con $d=6$): hasta 2 errores se corrigen; con **3 errores** el mensaje puede caer **a la misma distancia de dos palabras válidas** (detecta, pero no sabe a cuál corregir); con **4 o más** puede caer dentro de la esfera de otra palabra y **corregir mal** (miscorrección). Por eso hay que analizar el canal antes de elegir el código.

Cota de Singleton: $d_{min} \leq n-k+1$. De acá: la capacidad de detección es a lo sumo $n-k$ y la de corrección a lo sumo $\lfloor (n-k)/2 \rfloor$. Los códigos que cumplen la igualdad son **MDS** (los Reed-Solomon).

1.3 Bit de paridad simple

Se agrega un bit para que la cantidad de unos sea par (paridad par) o impar. Ejemplo: `001011` tiene tres unos → con paridad par se agrega `1` (`0010111`), con paridad impar `0` (`0010110`). **Detecta solo cantidades impares de errores**; $d_{min}=2$: detecta 1, no corrige.

1.4 Códigos pesados: base 37, ISBN-10 e ISBN-13

Son códigos para errores tipográficos humanos. Cada símbolo se **pondera por su posición** y se agrega un símbolo de control para que la suma ponderada sea divisible por la base.

Código base 37 (10 dígitos con pesos 0–9, 26 letras $A=10 \ldots Z=35$, espacio $=36$): se agrega P tal que $(S+P) \bmod 37 = 0$, o sea $P = 37 - (S \bmod 37)$ (y $P=0$ si $S \bmod 37 = 0$).

ISBN-10 (base 11 → dígitos 0–9 y `X`). Posiciones 1.º idioma, 2.º–3.º editorial, siguientes 6 el libro, 10.º símbolo de control. Debe cumplirse:

$$(10x_1 + 9x_2 + 8x_3 + 7x_4 + 6x_5 + 5x_6 + 4x_7 + 3x_8 + 2x_9 + 1 \cdot x_{10}) \bmod 11 = 0$$

El control se calcula como $x_{10} = 11 - \left(\sum_{i=1}^{9} (11-i)x_i \bmod 11 \right)$ (si da 11 → `0`, si da 10 → `X`).

- Ejemplo de cátedra: `0-13-065432-?` : suma $= 123$; $123 \bmod 11 = 2$; $x_{10} = 11-2 = 9$ → **`0-13-065432-9`**.

- **Capacidad:** $d_{min} = 2$. **Detecta un error** en cualquier posición (cambiar un dígito cambia la suma módulo 11 porque 11 es primo). **No corrige** (si sabés que el error está en una posición **conocida**, como un dígito faltante `?`, sí podés reponerlo). Con ≥2 errores la suma puede dar 0 y el error pasa inadvertido: es el *error indetectable* que pregunta la teórica.
- Por ser base **prima**, también detecta el **intercambio de dos dígitos adyacentes**.

ISBN-13 (versión actual). Pesos **1, 3, 1, 3, …**, módulo **10**:

$$\left(\sum_{i=1}^{13} w_i x_i\right) \bmod 10 = 0, \qquad w_i = 1 \ (i \text{ impar}),\ 3 \ (i \text{ par}).$$

Como 10 **no es primo**, detecta cualquier error de un dígito pero **no detecta** el intercambio de dos dígitos adyacentes cuando difieren en 5. Un ISBN inválido **se puede corregir solo si se asume cuál dígito está mal** (típicamente el de control): con el resto de dígitos fijos, el control correcto es $(10 - S_{12} \bmod 10) \bmod 10$. No hay forma de saber *cuál* dígito falló.

Por qué siempre una base prima (teórica fija): si el módulo N es primo, **todo número no nulo tiene inverso multiplicativo** y único, el sistema decodifica de forma **unívoca** (no hay dos mensajes posibles para un mismo recibido), no hay divisores de cero y el espacio de claves/valores se usa por completo. Con módulo compuesto algunos números no tienen inverso (en $\mathbb{Z}_{27}$ no lo tienen 3, 6, 9, 12, …).

1.5 Arreglos geométricos (gráficos)

Se disponen los bits de información en 2 o más dimensiones y se calculan paridades por **filas y columnas**; el chequeador de fila y el de columna que fallan **se cruzan en el bit erróneo**, que se invierte.

- **Código rectangular** (m filas × n columnas de información):
  - **Sin bit esquina:** $d_{min} = 3$ (detecta 2, corrige 1). Longitud $(m+1)(n+1) - 1$, chequeadores $m + n$. La redundancia es mínima cuando el arreglo es **casi cuadrado**.
  - **Con bit esquina** (paridad de las paridades): $d_{min} = 4$ (corrige 1 y detecta 2), longitud $(m+1)(n+1)$, un chequeador más.
- **Código triangular:** disposición en «escalera invertida»; cada chequeador controla una fila y una columna. Sigue con $d_{min} = 3$ pero con **~40 % menos de redundancia** que cualquier rectangular. *(Estructura: m chequeadores y $m(m-1)/2$ bits de información; cada bit de información cae en dos chequeadores. Esta fórmula la derivé del esquema; la diapositiva de la cátedra no se leyó bien en la extracción.)*
- **Código cúbico:** las tres dimensiones; cada chequeador es la paridad lógica de **una «cara»** del cubo. Un bit de información queda controlado por 3 chequeadores.
- **Código producto** (lo que preguntó el parcial 2025): si las filas forman un código de distancia d_1 y las columnas uno de distancia d_2, el código rectangular resultante tiene

  $$d_{min} = d_1 \cdot d_2.$$

  Con filas = Hamming ($d_1 = 3$) y columnas = paridad simple ($d_2 = 2$): $d_{min} = 6 \rightarrow$ corrige 2, detecta 5. **Cómo se corrige:** la paridad por columnas indica *qué columnas* fallan; el Hamming de cada fila indica *dónde, dentro de la fila*.

1.6 Aproximación geométrica: cuántos bits de control hacen falta

Para corregir t errores, las esferas de radio t alrededor de las 2^k palabras válidas **no deben superponerse**. El volumen de una esfera es $V = \sum_{i=0}^{t} \binom{n}{i}$ (palabras a distancia $\leq t$ de un centro), y como entran en el espacio de 2^n palabras:

$$2^n \ \geq \ 2^k \cdot \sum_{i=0}^{t} \binom{n}{i} \quad \Longleftrightarrow \quad 2^{\,n-k} \ \geq \ \sum_{i=0}^{t} \binom{n}{i} \qquad \text{(cota de Hamming)}$$

Para corrección simple ($t = 1$): $2^{n-k} \geq 1 + n$. Con $k = 4$: el n mínimo es **7** → Hamming (7,4). Con $k = 11$: $n = 15$ → Hamming (15,11). Cuando se da la **igualdad** el código es **perfecto**: las esferas llenan todo el espacio sin huecos (Hamming).

1.7 Código de Hamming

Es el corrector simple de **menor redundancia posible**; es un código **perfecto**, $d_{min} = 3$. Su idea: que los chequeadores sean **independientes** y trabajen lo mismo.

Posiciones: se numeran 1, 2, 3, … Los **chequeadores van en las potencias de 2** (1, 2, 4, 8, …) y la información en las demás (3, 5, 6, 7, 9…). Cada chequeador C_j es **paridad par** de las posiciones cuyo bit j en binario vale 1:

Chequeador	Posición	Controla	Ecuación en Hamming (7,4)
C_1	1	1, 3, 5, 7	$C_1 = I_1 \oplus I_2 \oplus I_4$
C_2	2	2, 3, 6, 7	$C_2 = I_1 \oplus I_3 \oplus I_4$
C_3	4	4, 5, 6, 7	$C_3 = I_2 \oplus I_3 \oplus I_4$

(donde I_1, I_2, I_3, I_4 ocupan las posiciones 3, 5, 6, 7.)

Síndrome: (S_1, S_2, S_3) se recalcula en el receptor (cada S_j es la paridad de su grupo **incluyendo** al chequeador). Si es $(0,0,0)$ no hay error. Si no, **leído como número binario es la posición del bit erróneo** (ej.: $(1,1,0) \rightarrow$ posición 6). Esa es la gracia de la numeración de Hamming.

Hamming extendido (8,4): se suma un bit de paridad global → $d_{min} = 4$: como detector detecta 3; como corrector corrige 1 y detecta 2.

Código	d_{min}	Detector	Corrector
Hamming (7,4)	3	2 errores	corrige 1
Hamming (8,4)	4	3 errores	corrige 1, detecta 2

Códigos equivalentes: son los que se obtienen permutando columnas o símbolos; conservan largo, cantidad de palabras y d_{min}, o sea la **misma capacidad**.

1.8 Códigos de bloques lineales: matrices G y H

Son los códigos cuyos chequeadores son **combinaciones lineales** (mod 2) de los bits de información. Se resumen en dos matrices:

- **Generadora** $G_{k \times n} = [\, I_k \mid P \,]$ (forma sistemática: los primeros k bits de la palabra son la información). La palabra es $v = i \cdot G$. P ($k \times (n - k)$) contiene los coeficientes de las combinaciones lineales.
- **Chequeadora** $H_{(n-k) \times n} = [\, P^T \mid I_{n-k} \,]$. Toda palabra válida cumple $v \cdot H^T = 0$.

Síndrome $s = r \cdot H^T$. El síndrome es la **combinación lineal de las columnas de H correspondientes a las posiciones con error**:

- 0 errores → $s = 0$.
- 1 error en la posición $i \rightarrow s =$ columna i de H.
- 2 errores en $i, j \rightarrow s =$ columna $i \oplus$ columna j.
- w errores → suma de w columnas.

Patrón de error $e = r \oplus v$ (los unos marcan los bits cambiados). Hay 2^n patrones pero solo 2^{n-k} síndromes, por eso hay que cumplir la **cota de Hamming** $\sum_{i=0}^{t} \binom{n}{i} \leq 2^{n-k}$.

Distancia mínima de un lineal = peso mínimo de una palabra no nula (la palabra nula es el «ancla»: el código es simétrico y la distancia de cualquier palabra al resto se ve igual que desde la nula).

Relación G–H (teórica fija): son **ortogonales**: $G \cdot H^T = 0$. Si $G = [I \mid P]$ entonces $H = [P^T \mid I]$; si $G = [P \mid I]$ entonces $H = [I \mid P^T]$. Cada fila de H define una ecuación de paridad; multiplicar una palabra por H^T da 0 si es válida y distinto de 0 si hubo error.

1.9 Códigos cíclicos

Son lineales con la propiedad: **toda rotación de una palabra válida es válida**. Se trabajan con polinomios sobre $GF(2)$: la cadena $(b_0, b_1, \ldots, b_{n-1})$ es $b_0 + b_1 x + \ldots$. Rotar equivale a multiplicar por x módulo $x^n + 1$.

Polinomio generador $g(x)$. Todo cíclico (n, k) tiene un único polinomio de grado mínimo, el generador. **Propiedades (teórica fija del parcial):**

1. $g(x)$ es de **grado** $n - k$.
2. $g(x)$ es **factor de** $x^n + 1$.
3. Tiene coeficiente 1 en el término independiente y en el de mayor grado.
4. Es el polinomio de **menor grado** del código (y primo, según la cátedra).
5. Toda palabra válida $V(x)$ es **múltiplo de** $g(x)$ y todo múltiplo de $g(x)$ (de grado $< n$) es palabra válida.
6. **Toda rotación** de una palabra válida es otra palabra válida.

Polinomio de chequeo: $h(x) = \dfrac{x^n + 1}{g(x)}$ (grado k). Los parciales dan $h(x)$ (a veces lo llaman «polinomio generador» por error) y piden $g(x)$: $g(x) = (x^n + 1)/h(x)$.

Codificación:

- **No sistemática:** $V(x) = i(x) \cdot g(x)$. Rápido pero la información queda mezclada.
- **Sistemática (la preferida):** $r(x) = \left(x^{n-k}\, i(x) \right) \bmod g(x)$ y $V(x) = x^{n-k} i(x) + r(x)$. La información queda intacta y los chequeadores son $r(x)$.
  - Con la convención de los parciales («mayores potencias a la derecha») los chequeadores ocupan las posiciones $0 \ldots n-k-1$ y la información las $n-k \ldots n-1$, así que la matriz queda $G = [P \mid I_k]$.
  - Ejemplo (Hamming (7,4), $g = x^3 + x + 1$, información $x^2 + 1$): no sistemática $= (x^2+1)(x^3+x+1) = x^5 + x^2 + x + 1$; sistemática: $x^3(x^2+1) = x^5 + x^3$, resto $x^2 \rightarrow V = x^5 + x^3 + x^2$. *(En este ejemplo de cátedra se escribe con la mayor potencia a la*

izquierda.)

Decodificación: se divide $R(x)$ por $g(x)$; el **resto es el síndrome**. Resto 0 → palabra válida; resto $\neq 0$ → hubo error. La información se lee de los k bits de la parte de información (en la forma sistemática).

Matrices:

- G **(no sistemática):** las k filas son $g(x)$ desplazada $0, 1, \ldots, k-1$ posiciones (se completa con ceros hasta n bits).
- H**:** se arma con el **polinomio de chequeo** $h(x)$ **con los coeficientes en ORDEN INVERSO**, desplazado $0, 1, \ldots, n-k-1$ posiciones (es una matriz de $(n-k) \times n$). *Si usás* $h(x)$ *sin invertir,* H *no es ortogonal a* G (verificado; es un error de la resolución de alumno de 2025, sección 12).
- Alternativa equivalente: forma sistemática $G = [P \mid I]$ con P las filas $x^{n-k+i} \bmod g(x)$ y $H = [I \mid P^T]$.

Cómo se ve el error en el síndrome (cíclicos): para un error simple en x^i, el síndrome es $x^i \bmod g(x)$. Si $i < n-k$ el síndrome es el propio x^i (por eso en la resolución 2025 «el resto x^2 me dice que el error está en x^2»); si $i \geq n-k$ hay que reducir módulo $g(x)$. Los errores dobles se buscan comparando contra la suma de dos de esos síndromes (ver 5.2, apartado d).

1.10 Circuitos con registros de desplazamiento (LFSR)

La codificación de un cíclico se implementa con un **registro de desplazamiento** y compuertas XOR que calculan «al vuelo» el resto de la división. Hay dos arquitecturas: una con $n-k$ **celdas** (conexiones dadas por los coeficientes de $g(x)$) y otra con k **celdas** (conexiones dadas por $h(x)$). Regla de lectura: los bits se mueven a la derecha; si hay una compuerta XOR hay que hacer la cuenta, si los bloques son contiguos el bit pasa tal cual.

Aviso de Fabricio: hay dos tipos de LFSR: **Fibonacci** (se usa en criptografía) y **Galois** (se usa en control de error). Para usar uno de Fibonacci con un código cíclico hay que usar el **polinomio inverso**; los gráficos de las presentaciones muestran el de Fibonacci aunque el código use el de Galois. La semilla no puede ser todo ceros.

Para «dibujar el circuito de $h(x) = x^3 + x^2 + x + 1$» (2010): registro de $k = 3$ **celdas** (el grado de h) con realimentación por XOR de las celdas cuyos coeficientes de $h(x) = 1 + x + x^2 + x^3$ valen 1 (acá, las tres). En el parcial se pide el dibujo a mano; lo que se evalúa es que **las conexiones coincidan con los coeficientes** del polinomio. *(Verificá el sentido de las conexiones con la cátedra: depende de si el diagrama es de Fibonacci o de Galois.)*

1.11 BCH y Reed-Solomon, campos de Galois

- **BCH:** familia de cíclicos que corrigen cualquier combinación de t errores (Hamming corrige solo 1). Introducen la **distancia de diseño**, que asegura matemáticamente d_{min}; muy útiles contra ráfagas.
- **Reed-Solomon (RS):** subconjunto de los BCH que opera sobre **símbolos** (bytes) y no sobre bits. Por eso es muy potente contra **ráfagas**: da igual que un byte tenga 1 o 8 bits dañados, cuenta como **un símbolo erróneo**.

Campo de Galois $GF(q)$**,** $q = 2^m$**.** Se construye con un **polinomio irreducible** de grado m (no es todavía el generador del RS: solo genera los elementos del campo), que define el elemento primitivo α; sus potencias $\alpha^0, \alpha^1, \ldots, \alpha^{q-2}$ recorren todos los no nulos, con $\alpha^{q-1} = 1$.

Parámetros RS(n, k, t) **sobre** $GF(2^m)$**:**

Magnitud	Valor
Longitud de la palabra (en símbolos/bytes)	$n = q - 1 = 2^m - 1$
Bits por símbolo	m
Grado del polinomio irreducible del campo	m
Símbolos de paridad (chequeadores)	$n - k = 2t$
Grado del polinomio generador RS	$2t$
Símbolos de información	$k = n - 2t = n - \text{chequeadores}$
Capacidad de corrección	t símbolos
Distancia mínima (MDS)	$d_{min} = n - k + 1 = 2t + 1$

Generador: $g(x) = (x - \alpha)(x - \alpha^2) \cdots (x - \alpha^{2t})$. Ejemplo de cátedra RS(7,3,5) sobre $GF(8)$ ($x^3 + x + 1$): $g(x) = (x - \alpha)(x - \alpha^2)(x - \alpha^3)(x - \alpha^4) = x^4 + \alpha^3 x^3 + x^2 + \alpha x + \alpha^3$ (verificado). Codificación sistemática igual que cualquier cíclico, pero con la aritmética del campo extendido. Clásico: **RS(255,223)** con bytes: 32 bytes de paridad, corrige $t = 16$ bytes.

Tabla del parcial 2012 T2 (campo, grado del polinomio del campo, n, bits/byte, chequeadores, información, t, ráfaga con 7 códigos entrelazados). Reglas y las tres filas que da el enunciado, completadas:

Dato del enunciado	Campo	n	bits/byte	chequeadores $2t$	info k	t	Ráfaga con 7 entrelazados
ráfaga de **100 bytes** con 7 códigos	$GF(2048)$ ($m = 11$)	2047	11	30	2017	15	$7 \cdot 15 = 105 \geq 100$ ✓
grado del campo 9, **461 bytes** de información	$GF(512)$	511	9	50	461	25	175
$n = 127$, $t = 10$	$GF(128)$ ($m = 7$)	127	7	20	107	10	70

(Las filas 2 y 3 las interpreté desde los datos que quedaron en el enunciado; verificá cuál dato era cuál en tu copia.) Para la fila 1: la ráfaga pedida determina $t = \lceil 100/7 \rceil = 15$.

1.12 Entrelazamiento (intercalado)

Los códigos de bloque corrigen errores **aleatorios**; una ráfaga de 4 bits seguidos supera su capacidad. **Entrelazar L palabras**: se escriben en una matriz fila por fila y se **transmite por columnas** (123 456 789 → 147 258 369). Una ráfaga en el canal queda repartida en el receptor entre L palabras distintas, y cada una recibe pocos errores.

$$\text{ráfaga corregible} = L \cdot t \quad (\text{con } t \text{ la capacidad de corrección de cada palabra})$$

Ejemplos: $t = 2$, $L = 25 \rightarrow$ **50 bits** seguidos (2025); $t = 2$, $L = 20 \rightarrow 40$; $t = 3$, $L = 20 \rightarrow 60$; con 7 códigos Reed-Solomon, $7t$ **bytes**. *(Idea simétrica: la capacidad de detección de ráfaga también se multiplica por L.)* El código sigue siendo el mismo: lo único que cambia es la **nueva capacidad ante ráfagas**.

Concatenación en serie: código externo → entrelazador → código interno; el receptor decodifica en orden inverso. **Concatenación en paralelo (códigos turbo):** el entrelazador solo reordena la información y la secuencia final es información + redundancia 1 + redundancia 2. Los dos decodificadores **cooperan iterativamente** intercambiando probabilidades hasta converger.

1.13 Códigos convolucionales y turbo

Los **convolucionales** no segmentan en bloques: tienen **memoria** (un registro de desplazamiento) y cada bit de salida depende de los bits actuales y de una cantidad fija de anteriores. Los **códigos turbo** concatenan **en paralelo** dos o más codificadores convolucionales simples con un **entrelazador pseudoaleatorio** (una permutación fija elegida una vez) y decodifican iterativamente: logran operar muy cerca del **límite de Shannon** (los códigos aleatorios de Shannon eran óptimos pero inviables; el turbo emula ese comportamiento con complejidad razonable).

---

2. Criptografía: Unidad 7 y Temas 1–5 de la cátedra

2.1 Definiciones y pilares

Criptografía: rama de la Matemática y hoy de la Informática/Telemática que usa métodos y técnicas para cifrar y/o proteger un mensaje o archivo con un **algoritmo y una o más claves**. (La definición de la RAE —«arte de escribir con clave secreta»— es poco ajustada: hoy es una **ciencia**, se protegen archivos de todo tipo, hay sistemas de una o dos claves y la representación binaria es «enigmática» solo para los humanos.)

- **Criptología:** estudia todo lo relacionado: cifra + criptoanálisis. **Criptólogo:** crea algoritmos para proteger. **Criptoanalista:** rompe algoritmos, busca la clave o el texto en claro.
- **Texto en claro** M; **criptograma** C; **cifrar** (el término correcto; «encriptar» no existe en el diccionario) y **descifrar** (con la clave) vs. **criptoanalizar** (sin ella).

Pilares de la seguridad informática (la cripto interviene en todos menos la disponibilidad):

Pilar	Qué garantiza	Ejemplo
Confidencialidad	solo los autorizados acceden	cifrado
Integridad	los datos no fueron alterados	hash, firma
Disponibilidad	los datos están accesibles	redundancia de servidores
Autenticidad	quién dice ser quién	certificados
No repudio	nadie niega haber hecho una acción; **de origen** (el emisor no niega haber enviado) y **de destino** (el receptor no niega haber recibido)	firma digital

Mecanismos: *cifrado* (confidencialidad); *integridad de datos* (funciones hash); *firma digital* = datos + valor de control cifrado con una componente secreta del emisor → integridad + autenticación + no repudio; *control de acceso*; *intercambio de autenticación* (simple: usuario y contraseña; fuerte: clave pública, certificados).

2.2 Criptosistema

Es el conjunto de elementos que permite proteger información en un **canal inseguro** (todo medio de transmisión es inseguro por definición). Componentes:

- espacio de textos en claro **M**; espacio de criptogramas **C**; espacio de claves **K**;
- transformaciones de cifrado $E_k : M \to C$ y de descifrado $D_k : C \to M$, con $D_k(E_k(M)) = M$ (inversas, o claves inversas dentro de un cuerpo finito).

El algoritmo normalmente es **público**; solo la clave es secreta (principio de Kerckhoffs). *Recomendaciones de Bacon:* cifrar y descifrar es sencillo, imposible hallar M sin la clave, el criptograma debe parecer inocente. *Recomendaciones de Kerckhoffs:* en la práctica imposible de criptoanalizar, claves fáciles de recordar, sin larga lista de reglas.

Ataques (de menos a más trabajo para el atacante): solo con el criptograma; con texto claro conocido; con texto claro elegido; con texto cifrado elegido; búsqueda de combinaciones de claves (fuerza bruta).

2.3 Secreto perfecto de Shannon

Un sistema tiene **secreto perfecto** si conocer el criptograma **no da ninguna información** sobre el mensaje: $I(M;C) = 0 \iff H(M) = H(M \mid C) \iff P(M) = P(M \mid C)$. Condiciones (teórica fija):

1. **Clave tan larga como el mensaje** (espacio de claves $\geq$ espacio de mensajes).
2. Claves **equiprobables** (aleatorias).
3. **Uso único**: se usa una vez y se destruye (*one-time pad* / cifrador de Vernam).

Es **seguridad incondicional** (independiente de la capacidad del atacante). La práctica usa **seguridad computacional** (romperlo es inviable con los recursos actuales).

Teórica «cuánto debe valer $I(A;B)$ con A mensaje y B criptograma»: idealmente **0** (mínima: ninguna relación entre entrada y salida). En la práctica no es exactamente 0, pero cuanto **menor**, mejor.

Números grandes (Tema 2): 2^{56} claves de DES a un millón por segundo son **más de 2200 años** (fue roto en 1998 con esfuerzo compartido); con una clave de 128 bits (IDEA) son del orden de 10^{25} años. Recorrer un espacio de 256 bits a 1 millón/s: 2^{212} años. ($1\ \text{millón} = 2^{20}$, $1\ \text{año} \approx 2^{25}$ s.)

2.4 Cifrados simétricos clásicos

Clasificaciones: *histórica* (clásicos vs. modernos, no técnica); *por el tratamiento* (bloque vs. flujo); *por la clave* (secreta/simétrico vs. pública/asimétrico). Los clásicos usan dos herramientas de Shannon: **sustitución** (confusión) y **transposición** (difusión).

- **Confusión:** transformación para **mezclar** los elementos y complicar la dependencia entre clave y criptograma. Se logra con **sustituciones**.
- **Difusión: dispersar** las propiedades estadísticas del lenguaje por todo el criptograma. Se logra con **transposiciones (permutaciones)**.

(Teórica fija: *«definir confusión y difusión; cuál algoritmo introduce cada uno»*.)

Todo el álgebra es **modular**: n = tamaño del alfabeto. En el alfabeto de 27 letras (`A=0,…,N=13,Ñ=14,O=15,…,Z=26`). El **inverso** a^{-1} de a cumple $a \cdot a^{-1} \equiv 1 \pmod{n}$ y **existe sii** $\gcd(a,n) = 1$ (si n es primo, existe siempre).

Transposiciones (difusión)

- **Escítala** (Esparta, siglo V a.C.): cinta enrollada en un bastón; **la clave es el diámetro**.
- **Transposición simple por bloques:** clave = permutación $K = (k_1, \ldots, k_N)$. El texto se parte en bloques de N; el j-ésimo carácter del criptograma es el k_j-ésimo del bloque: $C_j = M_{k_j}$. Ejemplo: $K = (3,1,4,2)$, `AHÍ VA LA BOMBA` → `IAVH AABL BOAM`. **Descifrar:** usar la **permutación inversa** K^{-1} (donde $K_j = i$ entonces $K^{-1}_i = j$).
- **Transposición filas/columnas:** se escribe el texto **por filas** en una matriz $M \times N$ y se lee **por columnas** en el orden que indica la clave. Ej.: matriz 3×4, clave $(4,2,3,1)$, `AHIVALABOMBA` → `VBA HLM IAB AAO`.

Importante: hay **dos convenciones** posibles para la clave (la de las presentaciones «leer la columna k_j en la posición j» y la inversa «la columna j se coloca en la posición k_j»). **El parcial 2012 usa la segunda** (las claves (4,1,3,2) y (2,4,1,3) solo dan texto con sentido así). **Probá las dos: la correcta da un texto con sentido.**

¿Una transposición de una transposición es más segura? (Práctico, pregunta 6.) Si las dos usan **el mismo largo de bloque** N, la composición de dos permutaciones de N elementos es **otra permutación de N elementos** (forman un grupo, S_N): **no** da más seguridad

ni más claves. Si los largos o las matrices son **distintos**, equivale a una única permutación de largo $\mathrm{mcm}(N_1, N_2)$, más compleja de atacar aunque sigue siendo una transposición; la seguridad mejora **solo en ese caso** (y no es una buena práctica en cualquier caso).

Sustituciones monoalfabéticas (confusión)

- **Polybios** (siglo II a.C.): cada letra se reemplaza por un par de coordenadas; duplica el tamaño del texto.
- **César:** $C_i = (M_i + 3) \bmod 27$; generalizado $C_i = (M_i + k) \bmod n$, $M_i = (C_i - k) \bmod n$. Cada letra se cifra **siempre igual** → vulnerable al análisis de frecuencias (la letra más frecuente del criptograma ≈ la `E` : $k = C_E - 4$).
  - Ejemplo del Práctico, `AL PAN PAN, Y AL VINO...` : con $k = 4 \rightarrow$ `EO TEQ TEQ, C EO ZMQS...` ; con $k = 18 \rightarrow$ `RC HRE HRE, P RC NZEG...` (alfabeto mod 27).
  - **¿Presenta estructura de grupo? Sí.** César forma un **grupo abeliano**: es cerrado (cifrar con k_1 y luego k_2 equivale a cifrar con $k_1 + k_2$), tiene identidad ($k = 0$), inverso ($-k$) y es asociativo. Consecuencia: **el cifrado múltiple no aumenta la seguridad**. Lo mismo ocurre con Vigenère (suma de claves, ver el ejemplo `PACO` + `CINE` = `RIOS`). **DES no es un grupo** y por eso el cifrado múltiple sí sirve.
  - Descifrado del Práctico, `YAMNMYOAYPY` con $k = 12$: `NOABANDONEN` .
- **Afín:** $C_i = (a \cdot M_i + b) \bmod n$, $M_i = (C_i - b) \cdot a^{-1} \bmod n$. Condición: $\gcd(a, n) = 1$. Cantidad de claves: $\varphi(n) \cdot n$; con n primo, $(n-1) \cdot n$ (para $n = 131$: 17030). Ataque: se asocian las dos letras más frecuentes del criptograma a `E` y `A` y se resuelve un sistema de 2 ecuaciones. *Ejemplo:* $E \rightarrow N$, $A \rightarrow F$: $(a \cdot 0 + b) = 5$, $(4a + 5) \equiv 13 \rightarrow a = 8 \cdot \mathrm{inv}(4, 27) = 8 \cdot 7 = 2$, $b = 5$.
- **Homofónica:** el alfabeto de salida tiene **más símbolos**; las letras frecuentes tienen **varias** representaciones (con más probabilidad cuanto más frecuentes), lo que frustra el análisis estadístico (`E` →9,14,15,21,22). Más cifras que letras.
- **Hill** (poligrámico, lineal): $C = K \cdot M \bmod n$ con K una matriz clave $N \times N$ con **inversa** en $\mathbb{Z}_n$, o sea $\gcd(|K|, n) = 1$; $K^{-1} = \mathrm{adj}(K)^T \cdot |K|^{-1}$. Seguro contra frecuencias pero **vulnerable a texto claro conocido** por **Gauss-Jordan** (la linealidad permite despejar K).
- **Trithemius:** clave progresiva, $C_i = (M_i + i) \bmod n$.

Vigenère (polialfabético)

Clave K de largo L que se repite: $C_i = (M_i + K_{i \bmod L}) \bmod n$, $M_i = (C_i - K_{i \bmod L}) \bmod n$. Una misma letra se cifra distinto según su posición; si la clave tiene más de ~6 letras distintas la distribución del criptograma se aplana. **No es seguro**: se rompe con:

- **Método de Kasiski:** repeticiones de cadenas de 3 o más caracteres en el criptograma → la distancia entre ellas es **múltiplo del largo de la clave**; el mcd de varias distancias es candidato a L. Se parte el criptograma en L subcriptogramas (cada uno es un César) y se ataca cada uno por frecuencias: regla **AEO** (la `E` está 4 lugares a la derecha de la `A` , la `O` a 15; la `S` a 19). Ejemplo de cátedra: mcd(256,104,72,156,32)=4 → clave `ABER` .
- **Índice de coincidencia:** $IC = \sum p_i^2$; en castellano $IC \approx 0{,}072$. Un subcriptograma monoalfabético tiene IC cercano a ese valor; si es $< 0{,}05$ probablemente sea polialfabético.

Hallar la clave con pareja claro/cifrado conocida (Práctico, ejercicio 4): $K_i = (C_i - M_i) \bmod n$; el patrón se repite con período = largo de la clave. Resultado: con alfabeto de 27 letras la clave es `COLOSSUS` (y se verifica en todo el mensaje; con 26 letras la clave no se repite, así que el alfabeto es el de 27).

Criptoanálisis del César por fuerza bruta (Práctico, ejercicio 7, `MXDQLWR` , candidatos `JORGITO` , `JUANITO` , `JAIMITO`): el desplazamiento tiene que ser **el mismo en todas las letras**: solo `JUANITO` da $k = 3$ en todas (con la `Ñ` o sin ella).

2.5 Entropía y criptosistemas (pregunta fija de cada parcial)

La entropía de orden 0 se mira con la fuente como **independiente** (frecuencia de cada símbolo); la de orden 1 tiene en cuenta el **símbolo anterior** (Markov), etcétera.

Algoritmo	Orden 0	Orden 1 y superiores	Por qué
Transposición (difusión)	**NO la modifica**	**la modifica**	mismos símbolos, otro orden: se conservan las frecuencias pero se rompen las dependencias entre vecinos
César / afín / sustitución monoalfabética	**NO la modifica**	(ver nota)	cada símbolo se reemplaza **siempre por el mismo**: es una permutación de símbolos, las probabilidades solo se reasignan
Vigenère (polialfabético)	**SÍ la modifica**	la modifica	un mismo símbolo se cifra distinto según su posición → las frecuencias se aplanan
RSA/afín con bloques de 1 carácter	no (es monoalfabético)	—	idem

Nota importante (verificalo con tu docente): una sustitución monoalfabética es una **relabelación biyectiva** de símbolos, así que conserva **todos** los órdenes de entropía, no solo el 0. La hoja de respuestas de la cátedra dice «RSA, afín y César **no modifican la entropía de orden 0**» y «las transposiciones **modifican la de orden 1 en adelante**». Por eso, a la pregunta del parcial 2025 *«¿qué*

algoritmo mantiene la entropía de orden 0 pero altera las de orden superior?» la respuesta consistente con la cátedra es la **transposición** (la resolución de alumno de 2025 contestó «sustitución monoalfabética», que además no altera los órdenes superiores; ver sección 12).

Ejemplo calculado (parcial 2010, 24 letras): texto en claro `SILORESOLVIERONAPROBARON` ($H_0 = 3{,}268$ bits, 11 símbolos distintos), tras Vigenère `RACING` ($H_0 = 3{,}887$, 16 símbolos distintos), tras la permutación $(6, 5, 4, 3, 2, 1)$ ($H_0 = 3{,}887$, **igual**: la permutación no cambia H_0). Redundancia de la fuente original con $H_{max} = \log_2 26 = 4{,}700$: $1 - 3{,}268/4{,}700 = 30{,}5\%$ (o $H_{max} - H = 1{,}43$ bits).

Teórica «¿qué fuente conviene comprimir?»: la **original**. Un texto cifrado se parece a ruido (entropía máxima), así que **no se comprime** (y las intermedias tienen la entropía ya subida por Vigenère). **Se comprime primero y se cifra después.** Por la misma razón, ordenando por entropía relativa: `.rar` **>** `.exe` **>** `.txt` (comprimir/cifrar sube la entropía; el texto plano es el de menor entropía).

2.6 Cifrado en bloque: Feistel, DES, 3-DES, IDEA, AES

Bloque vs. flujo: el cifrado en **bloque** aplica el mismo algoritmo a bloques de 64 o 128 bits (alta difusión, inmune a bloques extraños; lento y un error se propaga al bloque). El de **flujo** cifra elemento a elemento con un flujo de clave (rápido, resistente a errores; baja difusión y vulnerable a alteraciones por separado).

Cifrador de Feistel: el bloque de N bits se parte en dos mitades L, R; durante n vueltas se hace $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, k_i)$ con f unidireccional. Se descifra con **el mismo algoritmo** porque $Y \oplus X \oplus X = Y$. Ejemplo de cátedra en mod 27: sustitución $+1$ y permutación $(3, 2, 4, 1)$ en dos vueltas sobre `STAR WARS, LA MISIÓN CONTINÚA` → `SBTX BUST PJÑT NBJM VÑBJ ÑPUD`.

Algoritmos conocidos: Lucifer (128/128/16 vueltas, origen del DES), DES (64/56/16), Loki, RC2, CAST, Blowfish, IDEA (64/128/8), Skipjack (con puerta trasera), **Rijndael/AES** (128 bloque, 128+ clave).

DES (Data Encryption Standard):

- Historia: 1973 concurso del NBS, 1974 la NSA elige Lucifer con cambios, **1976 estándar**; certificado en 1987 y 1993; 1997 no se recertifica y se llama al concurso AES.
- **Bloque 64 bits; clave 64 bits de los cuales 56 efectivos** (8 de paridad: posiciones 8, 16, …, 64). Espacio $2^{56} \approx 7{,}2 \cdot 10^{16}$. La NSA recortó la clave de 128 de Lucifer.
- **16 vueltas**, estructura Feistel.
- Pasos: **permutación inicial IP** (sin interés criptográfico) → 16 vueltas → **IP⁻¹**. En cada vuelta: la mitad derecha R_{i-1} (32 bits) se **expande a 48** (permutación con expansión E) → **XOR** con la subclave k_i (48 bits) → **8 cajas S** (6 bits → 4 bits, función **no lineal** y unidireccional; es **lo que da la fortaleza**) → permutación P (32 bits) → XOR con L_{i-1}. La derecha anterior pasa tal cual como nueva izquierda.
- **Cajas S:** con los 6 bits $b_1b_2b_3b_4b_5b_6$, la **fila** es b_1b_6 (extremos) y la **columna** $b_2b_3b_4b_5$ (centrales). Ejemplo: entrada `101100` en S_2 → fila `10` =2, columna `0110` =6 → $S_2[2][6] = 13 =$ `1101`.
- **Subclaves:** PC-1 descarta los 8 bits de paridad (56 bits → 2 mitades C_0, D_0 de 28), rotación a izquierda de 1 o 2 bits según la vuelta (1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1: total **28**, por eso $C_{16} = C_0$), PC-2 elige 48 bits.
- **Descifrar:** mismo algoritmo con las subclaves **en orden inverso** $k_{16}, \ldots, k_1$.
- **Claves débiles** ($E_k(E_k(M)) = M$: `0101…`, `FEFE…`, `E0E0…F1F1…`, `1F1F…0E0E…`, ceros, unos) y **semidébiles** (6 pares).
- **Fin de DES (DES Challenge):** 1997 roto en 96 días con 80 000 PCs; ene-1998 en 39 días; jul-1998 el **DES Cracker** (EFF, US$ 200 000) en **56 horas**; ene-1999 en **22 h**. Hoy es vulnerable por la longitud de clave.

Modos de cifra (valen para cualquier bloque):

Modo	Descripción	Característica
ECB (libro electrónico de códigos)	cada bloque se cifra independiente con la misma clave	bloques iguales dan criptogramas iguales (permite reconstruir el «libro»; ataque por repetición, comienzos/finales fijos)
CBC (encadenamiento)	$C_i = E_k(P_i \oplus C_{i-1})$, con vector inicial $IV = C_0$ secreto de 64 bits; descifrado $P_i = D_k(C_i) \oplus C_{i-1}$	evita el ataque por repetición, enmascara el mensaje como un cifrador de flujo; un error se propaga a **dos bloques contiguos**
CFB, **OFB**	realimentación del criptograma / de la salida	convierten el bloque en flujo

Cifrado múltiple: si un sistema es un **grupo**, cifrar con k_1 y luego con k_2 equivale a una sola clave k_3 (no sirve). **DES no es un grupo.** Doble DES parece de 2^{2n} pero solo vale 2^{n+1} (ataque *meet-in-the-middle*). **Triple DES (EDE):** $C = E_{k_3}(D_{k_2}(E_{k_1}(M)))$ (se codifica con k_1, se decodifica con k_2 y se vuelve a codificar con k_3; con dos claves $k_3 = k_1$, compatible con DES simple) → **112 bits efectivos**; se usa en e-commerce (SET) y PGP. Otras variantes: subclaves independientes (768 bits, pero rompible por criptoanálisis diferencial), DES generalizado, S-cajas alternativas (nadie encontró mejores que las de la NSA, que ya resistían el criptoanálisis diferencial).

IDEA: bloque 64 bits, clave 128, **8 vueltas + transformación final**, 52 subclaves de 16 bits (6 por vuelta). Divide el bloque en 4 subbloques de 16 bits. **Tres operaciones** incompatibles entre sí (dan confusión): **XOR**, **suma mod** 2^{16} y **producto mod** $2^{16} + 1 = 65537$

(primo, asegura inversos). La **difusión** la da el bloque MA (multiplicación/adición). Se descifra con la misma estructura usando los **inversos** de las operaciones (XOR: él mismo; aditivo; multiplicativo). Inmune al criptoanálisis diferencial; usado en PGP.

AES (Rijndael, 2000, reemplazo de DES): *no* es Feistel; procesa el bloque entero en paralelo. Bloque **128 bits** (matriz de 4×4 bytes), clave **128/192/256** con **10/12/14** rondas, aritmética en $GF(2^8)$ (byte = polinomio). Resistente al criptoanálisis lineal y diferencial. *(Complemento: cada ronda aplica SubBytes, ShiftRows, MixColumns y AddRoundKey; la presentación solo da los parámetros.)*

RC2 (clave variable 8–1024 bits, sin cajas S) y **RC5** (de Rivest: bloque 32/64/128, vueltas 0–255, rotaciones dependientes de los datos) quedan como cultura general.

2.7 Cifrado de flujo y generadores de secuencia

Siguiendo a **Vernam (1917)**: cifra con **XOR** entre el mensaje y una **secuencia cifrante** S derivada de una clave secreta K compartida: $C = M \oplus S$, $M = C \oplus S$ (el descifrado es igual al cifrado por la **involución de XOR**). Lo importante es **cómo se genera** S. Como la clave tan larga como el mensaje no se puede intercambiar, se usa un **generador pseudoaleatorio** con una semilla corta (120–250 bits → períodos superiores a 10^{35}).

Condiciones de la secuencia cifrante: período largo; distribución uniforme de unos y ceros; rachas de dígitos (una o más ceros/unos entre dos distintos) bien distribuidas; imprevisible (computacionalmente intratable predecir el próximo bit).

Síncronos (la secuencia es independiente del texto; basta perder un bit para que el resto sea indescifrable) vs. **asíncronos** (dependen de los últimos t bits cifrados; se **resincronizan** solos).

Generador de congruencia lineal: $x_{i+1} = (a\,x_i + b) \bmod n$; a, b, n y la semilla x_0 son la clave. Ejemplo $a = 5, b = 1, n = 16, x_0 = 10$: **10, 3, 0, 1, 6, 15, 12, 13, 2, 11, 8, 9, 14, 7, 4, 5** (período 16). **Debilidad:** es lineal y por eso se ataca como un afín (con $b = 2$ el período cae a 8 y solo salen pares o impares; con $a = 4, b = 1$ la secuencia se clava en 5; con $a = 11, b = 1, x_0 = 7$ el período es 2). **Criptográficamente inútil.**

Registros de desplazamiento realimentados (FSR): n celdas; en cada pulso sale la celda de un extremo, las demás se desplazan y entra un bit nuevo calculado por una función de realimentación.

	NLFSR (no lineal)	**LFSR (lineal)**
Función	cualquier función booleana	**solo XOR** de celdas fijas
Período máximo	2^n (secuencia de De Bruijn)	$2^n - 1$ (**no** puede pasar por el estado todo ceros)
Dificultad cripto	alta	baja
Implementar / analizar	difícil	extremadamente fácil

LFSR: $a(t) = C_1 a(t-1) \oplus C_2 a(t-2) \oplus \cdots \oplus C_n a(t-n)$, con $C_n = 1$ y polinomio asociado $f(x) = C_n x^n + \cdots + C_1 x + 1$. Según el polinomio:

- **Factorizable:** sin interés criptográfico.
- **Irreducible:** el período es un divisor de $2^n - 1$, no el máximo.
- **Primitivo:** genera todos los estados no nulos; $T = 2^n - 1$ independiente de la semilla (**m-secuencia**). Es el de interés criptográfico.

Ejemplo de cátedra $f(x) = x^4 + x + 1$ (realimentación $S_1 = S_1 \oplus S_4$), semilla `1001`: la secuencia es `100100011110101` (15 bits, $T = 15$). **Debilidad (Berlekamp-Massey):** conocidos solo $2n$ **bits** de la secuencia se hallan las conexiones y se reconstruye **toda** la secuencia (un LFSR de 8 celdas con período 255: bastan 16 bits). Por eso en la práctica se **combinan varios LFSR con una función no lineal** (híbrido): p. ej. 3 LFSR de 11, 9 y 10 celdas, o los 3 LFSR del GSM.

Práctico, ejercicio 9 ($f(x) = x^5 + x^2 + 1$, semilla `10000`, cifrar `ENIGMA` en ASCII): realimentación $S_1 \leftarrow S_2 \oplus S_5$, salida $S_5 \rightarrow$ secuencia de **período** $2^5 - 1 = 31$: `00001010111011000111110011010 01` (convención de las presentaciones; otras convenciones —Galois, o salida por otra celda— rotan o reordenan la secuencia pero el período no cambia). `ENIGMA` en ASCII de 8 bits = `01000101010011100100100101000111010011010100 0001`; con los primeros 48 bits de la secuencia cifrante (que se repite) el criptograma es `010011111010001000110101100101010110001001 1001`.

2.8 Criptografía asimétrica (clave pública)

Cada usuario tiene **dos claves**: una **pública** (se publica) y una **privada** (no se comparte jamás). Lo que se cifra con una solo se descifra con la otra. Introducidos por **Diffie y Hellman (1976)**; dos usos:

Uso	**Se cifra con**	**Se descifra con**	**Garantiza**
Confidencialidad	clave **pública del receptor** E_B	privada del receptor D_B	solo el destinatario lee
Firma digital	clave **privada del emisor** D_A	pública del emisor E_A	**integridad + autenticidad + no repudio**
Ambos	$C = E_B(D_A(M))$	$M = E_A(D_B(C))$	confidencialidad e integridad **por separado**

Funciones unidireccionales con trampa: fáciles en un sentido, intratables en el inverso salvo con la trampa. Las más usadas:

- **Factorización:** $p \cdot q = n$ fácil; hallar p, q desde n es difícil (RSA).
- **Logaritmo discreto:** $\beta = \alpha^x \bmod n$ fácil; $x = \log_\alpha \beta$ difícil (Diffie-Hellman, ElGamal).
- Otras: **mochila**, **raíz discreta**.

La seguridad se mide por la **complejidad** del problema: para factorizar n, $e^{\ln n \cdot \ln \ln n}$ (no polinomial; igual el logaritmo discreto). Para n de 60 dígitos son $2{,}7 \cdot 10^{11}$ pasos; de 100 dígitos $2{,}3 \cdot 10^{15}$ (74 años a 1 paso/seg.); de 200 dígitos $1{,}2 \cdot 10^{23}$. Los sistemas asimétricos **no se rompen solo por fuerza bruta** sino por análisis matemático, por eso **una simétrica de 128 bits equivale a una asimétrica de ~2048 bits**.

Comparación simétrico vs. asimétrico (teórica de todos los años):

	Simétrico (clave secreta)	**Asimétrico (clave pública)**
Claves	**una** clave secreta compartida	**dos** claves (pública y privada) por usuario
Uso típico	**cifrar el mensaje**	**intercambio de clave de sesión y firma digital**
Velocidad	**muy alta** (cientos de MB/s en hardware)	**muy lenta** (cientos de KB/s; ~4000 veces más lenta)
Gestión de claves	**mala**: hacen falta $\frac{n(n-1)}{2}$ claves (n^2): 5 usuarios → 10	**buena**: solo la privada propia y $2n$ claves en total
Intercambio de claves	**problema principal**: hay que compartirlas por un canal seguro	**resuelto** (la pública es pública)
Longitud	~128 bits o más (hay que probar todo el espacio)	1024–2048 bits (se ataca matemáticamente)
Vida de la clave	corta (segundos)	larga (meses/año)
Firma digital	**no** (autentica el mensaje pero no al emisor de forma sencilla)	**sí** (autenticación y no repudio)
Confidencialidad e integridad	**simultáneas**: protegiendo la clave secreta	**por separado**: firmando con la privada y cifrando con la pública

Sistema híbrido (teórica fija). Combina lo mejor de ambos: el **mensaje** se cifra con una **clave de sesión simétrica** (rápida, p. ej. AES); la **clave de sesión** se cifra con la **clave pública del receptor** (confidencialidad e intercambio); el **hash** del mensaje se cifra con la **privada del emisor** (firma). El receptor usa su privada para recuperar la clave de sesión, con ella descifra el volumen de datos y con la pública del emisor verifica el hash. No es «más o menos seguro»: aporta características extra. Así funcionan los protocolos seguros de Internet.

2.9 Diffie-Hellman, RSA, ElGamal, firmas y hash

Diffie-Hellman (acuerdo de clave). No es un criptosistema: es un **protocolo para generar una clave compartida por un canal inseguro**; se basa en el logaritmo discreto.

1. A y B acuerdan públicamente un **primo grande** p (≥1024 bits) y un **generador** α (raíz primitiva módulo p).
2. A elige a secreto y envía $\alpha^a \bmod p$; B elige b y envía $\alpha^b \bmod p$.
3. A calcula $(\alpha^b)^a$, B calcula $(\alpha^a)^b$: el secreto compartido es $\alpha^{ab} \bmod p$ (luego se usa como clave simétrica).

Condiciones: p primo grande; $p-1$ con un factor primo grande (además del 2); α raíz primitiva (si no, α^i no recorre todos los restos y se facilita la fuerza bruta). *Ejemplo de cátedra:* $p = 1999$, $\alpha = 33$, $a = 47$, $b = 117$: A envía $33^{47} = 1343$, B envía $33^{117} = 1991$; ambos obtienen **1506** (verificado).

RSA (1978, Rivest-Shamir-Adleman). Se basa en la dificultad de **factorizar**.

1. Se eligen dos primos grandes p, q (secretos) y $n = p \cdot q$ (público).
2. $\varphi(n) = (p-1)(q-1)$.
3. **Clave pública** e con $\gcd(e, \varphi(n)) = 1$ (y $1 < e < \varphi(n)$).
4. **Clave privada** $d = e^{-1} \bmod \varphi(n)$ (algoritmo de Euclides extendido).
5. Se publican (n, e); se guardan d (y se pueden destruir p, q, φ).

Cifrar: $C = M^e \bmod n$ con $M \in [0, n-1]$ (clave pública del receptor). **Descifrar:** $M = C^d \bmod n$. **Firmar:** $C = M^{d_{emisor}} \bmod n_{emisor}$; verificar: $M = C^{e_{emisor}} \bmod n_{emisor}$. Ejemplo de cátedra: $n = 91 = 7 \cdot 13$, $\varphi = 72$, $e = 5$, $d = 29$, $M = 48$: $C = 48^5 \bmod 91 = 55$; $M = 55^{29} \bmod 91 = 48$. Para no calcular 55^{29} (51 dígitos) se usa **reducibilidad** (reducir cada paso) o el **algoritmo de exponenciación rápida** (*square and multiply*, ver 4.9).

Seguridad y precauciones: $n \geq 2048$ bits; p, q de ≥500 bits, **no cercanos** (si $p \approx q$, **Fermat** factoriza: $n = x^2 - y^2$, se busca $x > \sqrt{n}$ con $x^2 - n$ cuadrado perfecto, entonces $p = x + y$, $q = x - y$); $p-1$ y $q-1$ con factores primos grandes y $\gcd(p-1, q-1)$ pequeño; **no usar**

exponentes e muy bajos ni **módulo común** entre usuarios. **Mensajes no cifrables:** $M^e \equiv M \pmod{n}$ (el mensaje viaja «en claro»); siempre hay al menos 9 (con $n = 35, e = 11$: `{0,1,6,14,15,20,21,29,34}`, verificado). *Historia:* Clifford Cocks (GCHQ) lo descubrió en 1973 pero fue secreto.

ElGamal (logaritmo discreto; diseñado para firmas, luego para cifrado). **Claves:** primo grande p, generador α, privada $a < p$, pública (p, α, β) con $\beta = \alpha^a \bmod p$.

- **Cifrar** m: elegir k aleatorio **secreto, coprimo con** $p-1$; $r = \alpha^k \bmod p$, $t = m \cdot \beta^k \bmod p$ (el texto cifrado es el **par** (r, t) → el criptograma mide el **doble** que el original). **Descifrar:** $m = t \cdot r^{-a} \bmod p$.
- **Firmar** m: k coprimo con $p-1$ (**distinto en cada firma**); $r = \alpha^k \bmod p$; $s = \Big(H(m) - a\,r\Big) \cdot k^{-1} \bmod (p-1)$; firma $= (r, s)$. **Verificar:** $\beta^{\,r} \cdot r^{\,s} \equiv \alpha^{H(m)} \pmod{p}$.
- **Ejemplo verificado:** $p = 23$, $\alpha = 5$ (raíz primitiva), $a = 6 \to \beta = 8$. Cifrar $m = 10$ con $k = 7$: $r = 17$, $t = 5$; descifrar $5 \cdot 17^{-6} \bmod 23 = 10$ ✓. Firmar $H(m) = 14$ con $k = 5$: $r = 20$, $s = 14$; verificación $8^{20} \cdot 20^{14} \equiv 5^{14} \equiv 13$ ✓.
- **Por qué k no debe ser conocido ni reutilizado:** en las **firmas**, $a = (H(m) - k\,s)\,r^{-1} \bmod (p-1)$: si k se conoce o es fijo se despeja la clave **privada** a; si se reutiliza en dos mensajes, restando $s_1 - s_2$ se despeja k y de ahí a. En el **cifrado**, a nunca se despeja (queda en el exponente $\beta = \alpha^a$: habría que resolver el logaritmo discreto), pero si k se reutiliza se cumple $t_1/t_2 = m_1/m_2$: si se conoce un mensaje (cabecera, protocolo estándar) se obtiene el otro. $k = 1$ es el caso trivial: $t = m\beta$.

Criptografía de curva elíptica (ECC). ElGamal reformulado sobre **puntos de una curva** $y^2 = x^3 + ax + b$ con una operación de «suma» de puntos; $Q = kP$ es fácil pero hallar k dado P y Q es el **problema del logaritmo discreto en curvas elípticas**. Mismo nivel de seguridad con claves mucho más cortas.

Funciones resumen (hash). Transforman un mensaje de **cualquier largo** en una secuencia de **largo fijo** (huella digital). Propiedades de una función segura: **largo fijo**, **fácil de calcular**, **irreversible** (unidireccional) y **resistente a colisiones** (computacionalmente imposible hallar dos mensajes con el mismo resumen). Usos: verificar **integridad** y **firmas digitales** (no se cifra el documento entero, solo el hash con la privada del emisor).

- **MD5:** bloques de 512 bits, 4 registros de 32 bits (A,B,C,D), 4 rondas de 16 operaciones, resumen de **128 bits**; relleno (un 1, ceros, y 64 bits con el largo original) hasta múltiplo de 512. Se encontraron colisiones prácticas: **no recomendado**.
- **SHA-1:** bloques de 512 bits, 5 registros, 4 rondas de 20 operaciones, resumen de **160 bits** (colisiones encontradas después). **SHA-0** fue retirado. **SHA-256** es mucho más seguro.

Aritmética modular que hay que dominar (todo se calcula a mano):

- **Euler:** $\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right)$; para primo p: $\varphi(p) = p - 1$. $\varphi(40) = 40 \cdot \frac{1}{2} \cdot \frac{4}{5} = 16$; $\varphi(27) = 18$; $\varphi(35) = 24$; $\varphi(33) = 20$.
- **Inverso:** $a^{-1} = a^{\varphi(n)-1} \bmod n$ (Euler) o por **Euclides extendido**. Ej.: $13^{-1} \bmod 40 = 13^{15} \bmod 40 = 37$; $139^{-1} \bmod 1\,000\,003 = 107\,914$.
- **Existencia:** $\gcd(a, n) = 1$. Los conjuntos reducidos de restos de $\mathbb{Z}_{40}$: $\{1, 3, 7, 9, 11, 13, 17, 19, 21, 23, 27, 29, 31, 33, 37, 39\}$.

---

3. Imágenes: Unidad 10 (lo que sale en el parcial)

Pregunta corta que aparece en 2012, 2020 y 2025.

Adquisición: sistema óptico → sensor (señal eléctrica analógica) → digitalizador (A/D). Una imagen continua es una función $f(x, y) = i(x, y) \cdot r(x, y)$, **iluminación** $\times$ **reflectancia** $(0 < r < 1)$. **Discretización:** *espacial* (muestreo, matriz de $M \times N$ **píxeles**) y *en amplitud* (cuantización a L niveles; $L = 256$ con 8 bits). Color: **RGB** (24 bits = True Color, 16,7 millones de colores) o **indexado** (paleta; GIF, PCX, BMP de 8 bits); espacios luminancia/crominancia (YC_bC_r, YUV): el ojo tolera pérdida en la crominancia, base de JPEG/MPEG.

Operaciones puntuales: aritméticas (suma: reduce ruido promediando capturas; resta: elimina fondo, p. ej. angiografía; multiplicación/división: corrige iluminación no uniforme), lógicas (AND, OR, XOR, NOT con máscaras binarias), **negativo** $(255 - I)$, **umbralización**, traslación, rotación.

Operaciones espaciales (ventana, máscara, convolución): el píxel de salida es la combinación lineal de los vecinos de una ventana (3×3, 5×5) con los **pesos de la máscara**.

Filtro	Efecto	Máscara / método
Paso bajo (suaviza, elimina altas frecuencias, **emborrona** bordes)	reduce ruido	**Media** (coeficientes $1/m^2$, más grande = más desenfoque), **Gaussiano** (peso decrece con la distancia), **Mediana** (no lineal: ordena los niveles de la ventana y toma el central; **el más eficaz contra ruido «sal y pimienta»**)
Paso alto (realza bordes y detalles)	afila	máscara con centro positivo y periferia negativa; **la suma de los coeficientes es 0** (respuesta nula en zonas constantes); **high-boost** conserva parte de las bajas frecuencias
Paso banda	restauración	elimina frecuencias fuera de un rango (ruido aislable)

Filtro	Efecto	Máscara / método
Homomórfico	ruido multiplicativo / iluminación desigual	pasar a **logaritmos** para convertirlo en aditivo
Laplaciano	detección de bordes	p. ej. $\begin{pmatrix}0&1&0\\1&-4&1\\0&1&0\end{pmatrix}$

Ejemplos: filtro de **mediana** sobre la ventana $\begin{pmatrix}3&18&73\\1&7&20\\9&25&14\end{pmatrix}$: ordenados $1,3,7,9,14,18,20,25,73 \to$ mediana **14**; el centro 7 pasa a 14 (elimina el pico 73). **Media aritmética** (2012 T2): las dos ventanas 3×3 de la imagen 3×4 suman 1241 y $1243 \to$ centros suavizados $11 \to 137{,}9 \approx 138$ y $21 \to 138{,}1 \approx 138$.

Detección de rectas con máscaras (2012 T1): se aplica cada máscara al 3×3 y gana la de **mayor respuesta**: horizontal $\begin{pmatrix}-1&-1&-1\\2&2&2\\-1&-1&-1\end{pmatrix}$, vertical $\begin{pmatrix}-1&2&-1\\-1&2&-1\\-1&2&-1\end{pmatrix}$, 45° $\begin{pmatrix}-1&-1&2\\-1&2&-1\\2&-1&-1\end{pmatrix}$, 135° $\begin{pmatrix}2&-1&-1\\-1&2&-1\\-1&-1&2\end{pmatrix}$. Sobre $\begin{pmatrix}19&3&21\\4&19&11\\16&16&19\end{pmatrix}$ las respuestas son horizontal -26, vertical -14, 45° 40, **135°** $43 \to$ recta a **135°** (la diferencia con 45° es chica; la numeración de los ángulos depende de la convención del eje).

Histograma: *estiramiento* (más contraste), *contracción*, *desplazamiento* (offset), **ecualización** (4 pasos: suma acumulada → normalizar por el total de píxeles → multiplicar por el nivel máximo y redondear → mapear).

Vectorial vs. bitmap (2020, complemento mío): la imagen **vectorial** se describe con figuras matemáticas (puntos, curvas, rellenos), escala sin pérdida; la **bitmap (mapa de bits)** es una matriz de píxeles, ocupa más y se pixela al ampliarla. **RLE de PCX:** los valores `129,129,129,129,129,194` → una racha de 5 unos 129 se escribe con un byte de cuenta $192+5=197$ seguido del valor (`197, 129`); y el `194`, como es ≥ 192 y aparece una sola vez, necesita un byte de escape (`193, 194`). *(Complemento: el esquema exacto de PCX no está en los apuntes; verificalo.)*

---

4. Recetario: cómo se hace cada ejercicio

4.1 Hallar k, $g(x)$ y capacidad de un cíclico dado $h(x)$

1. $k = \deg h(x)$.
2. $g(x) = \dfrac{x^n+1}{h(x)}$ (división de polinomios sobre $GF(2)$; la resta es XOR).
3. Generá las 2^k palabras $V(x) = i(x)\cdot g(x)$ y buscá el **peso mínimo** ≠ 0 → d_{min}. *Atajo:* $d_{min} \leq$ cantidad de términos de $g(x)$.
4. $t = \lfloor (d_{min}-1)/2 \rfloor$ y detecta $d_{min}-1$. Redundancia $(n-k)/n$.

(Si el enunciado da $G(x)$ en bits, como en 2025, $n-k=\deg G$ y $k = n-\deg G$.)

4.2 Matrices G y H

- G: k filas = $g(x)$ en bits desplazada $0,1,\ldots,k-1$ y rellena de ceros hasta n.
- H: $h(x)$ **invertido** (de mayor a menor potencia), desplazado $0,\ldots,n-k-1$; es de $(n-k)\times n$.
- **Prueba de que está bien:** $G\cdot H^T = 0$ (mod 2).

4.3 ¿De qué bits de información depende cada chequeador?

Con la forma sistemática ($G=[P\mid I]$, convención de los parciales): cada fila i de P es $x^{n-k+i} \bmod g(x)$; el chequeador j vale el XOR de los i_k cuya fila de P tiene un 1 en la columna j.

- **(12,3)**, información i_0,i_1,i_2 (posiciones 9, 10, 11): P filas `110011001`, `101010101`, `100110011`. Chequeadores en las posiciones 0 a 8: $c_0=c_4=c_8=i_0\oplus i_1\oplus i_2$; $c_1=c_5=i_0$; $c_2=c_6=i_1$; $c_3=c_7=i_2$. (El código repite tres veces la paridad total y los datos.)
- **(14,4)**, información $i_0,\ldots,i_3$ (posiciones 10 a 13): P filas `1101000110`, `0110100011`, `1110010111`, `1010001101`. Chequeadores 0 a 9: $c_0=c_7=i_0\oplus i_2\oplus i_3$; $c_1=c_8=i_0\oplus i_1\oplus i_2$; $c_2=c_9=i_1\oplus i_2\oplus i_3$; $c_3=i_0$; $c_4=i_1$; $c_5=i_2$; $c_6=i_3$.

4.4 Validar $R(x)$, hallar el error y corregir

1. Dividí $R(x)$ por $g(x)$. **Resto 0 → válida.**
2. Si no, el **resto es el síndrome** $s(x)$.
3. Si $s(x)$ es una potencia x^i con $i<n-k$ (o coincide con $x^i \bmod g$ de alguna posición) → error simple en esa posición. Tabla de síndromes de error simple del (14,4) (bits x^0 primero): $x^0..x^9 \to$ ellos mismos; $x^{10}\to 1101000110$; $x^{11}\to 0110100011$; $x^{12}\to 1110010111$; $x^{13}\to 1010001101$.

4. Si no coincide con ningún simple, probá **sumas de 2** (y de 3 si $t = 3$) síndromes simples: el patrón de error $e(x)$ es el que da el mismo síndrome. Corregida: $R(x) + e(x)$ y comprobá que el resto da 0.

4.5 Entrelazado

$$\text{ráfaga corregible} = L \cdot t \quad (\text{bits o bytes según el código}).$$

Despejá L si piden «cuántos hay que entrelazar para una ráfaga B»: $L = \lceil B/t \rceil$.

4.6 Descifrar César / afín

1. $a^{-1} = a^{\varphi(n)-1} \bmod n$ (o Euclides).
2. $M_i = (C_i - b) \cdot a^{-1} \bmod n$. En el Práctico el profesor escribe $D(c) = a^{-1}c + b'$ con $b' = -a^{-1}b \bmod n$ («inverso» de b): son equivalentes.
3. **Ojo con el alfabeto:** el tamaño n y el orden de los símbolos los da el enunciado (27 letras; 29 con `$` y `#`; 40 con dígitos, espacio, `/` y `,`). · Ver Anexo 13 (pág. 23): inverso modular con Euler y con tabla de Euclides, y casos de parcial.

4.7 Descifrar un cifrador compuesto (Vigenère + transposición)

1. Si el enunciado dice *«primero Vigenère y luego transposición»*, **para descifrar se deshace primero la transposición y después Vigenère** (orden inverso).
2. Transposición: armá la matriz (por filas o por columnas, según diga) y probá las **dos convenciones de clave** (la que da texto con sentido es la buena).
3. Vigenère: restá módulo n la clave repetida.
4. **Cuidado con la numeración** $A = 1, \ldots, Z = 26$ (parcial 2010): con $A = 1$ la suma da un corrimiento adicional de 1; si el resultado casi tiene sentido pero «corrido una letra» es esto. *Verificado:* `SILORESOLVIERONAPROBARON` cifrado con `RACING` con $A = 1$ y permutación $(6, 5, 4, 3, 2, 1)$ da exactamente `LFXOJKLWEOPKYDJQPJUCADCG`.

4.8 Entropía de cada etapa y qué orden modifica cada algoritmo

Calculá $H = -\sum p_i \log_2 p_i$ de **cada** texto (original, tras el primer algoritmo, tras el segundo). La transposición **no cambia** H_0; Vigenère **sí**. Completá con la tabla de 2.5.

4.9 RSA a mano

- **¿Es válida la clave pública e?** $\gcd(e, \varphi(n)) = 1$ y $1 < e < \varphi(n)$ (si no, no hay d).
- **d = inverso de e módulo $\varphi(n)$** por Euclides extendido. Ejemplo: $\gcd(72, 5) = 1$, $-2 \cdot 72 + 29 \cdot 5 = 1 \rightarrow d = 29$.
- **Exponenciación rápida** (*square and multiply*): se recorren los bits del exponente; por cada bit se eleva al cuadrado y, si el bit es 1, se multiplica por la base; siempre se reduce módulo n. Ejemplo $55^{29} \bmod 91$, $29 = 11101_2$: bits `1,1,1,0,1` → 55, 27, 55, 22, **48** ✓.
- **Firmar y verificar:** firmar con d propio, verificar con e del emisor.
- **Romper un RSA con n chico:** factorizar n (Fermat si $p \approx q$), $\varphi = (p-1)(q-1)$, $d = e^{-1} \bmod \varphi$.

4.10 LFSR

1. Convertí $f(x)$ en conexiones: $f(x) = x^n + C_{n-1}x^{n-1} + \cdots + C_1 x + 1$ → la realimentación es el XOR de las celdas con $C_i = 1$ (y siempre $C_n = 1$).
2. Partí de la semilla, en cada pulso salida = celda de salida, desplazá y metés el bit XOR.
3. Período: hasta repetir la semilla; si f es **primitivo**, $T = 2^n - 1$.
4. Cifrar: XOR bit a bit del mensaje (ASCII de 8 bits) con la secuencia.

---

5. Ejercicios resueltos (todo verificado por computadora)

5.1 Parcial 17-11-2025, Tema 1 (cíclico (14,4), ISBN-13, producto, afín $n = 40$)

1) Cíclico (14,4,d_{min}), $G(x) =$ `11010001101` (mayores potencias a la derecha), $\deg G = n - k = 10$, o sea $g(x) = 1 + x + x^3 + x^7 + x^8 + x^{10}$.

a) Capacidad. Las 16 palabras $V = i \cdot g$ (bits x^0 primero, información → palabra → peso):

$i(x)$	palabra	peso		$i(x)$	palabra	peso
`0000`	`00000000000000`	0		`0001`	`00011010001101`	6
`1000`	`11010001101000`	6		`1001`	`11001011100101`	8
`0100`	`01101000110100`	6		`0101`	`01110010111001`	8
`1100`	`10111001011100`	8		`1101`	`10100011010001`	6

$i(x)$	palabra	peso		$i(x)$	palabra	peso
`0010`	`00110100011010`	6		`0011`	`00101110010111`	8
`1010`	`11100101110010`	8		`1011`	`11111111111111`	**14**
`0110`	`01011100101110`	8		`0111`	`01000110100011`	6
`1110`	`10001101000110`	6		`1111`	`10010111001011`	8

Pesos posibles: **6, 8 y 14** (nunca 7 ni 9). $d_{min} = 6 \rightarrow$ **corrige** $t = \lfloor 5/2 \rfloor = 2$; **detecta 5**.

b) G **y** H**.** G (4×14), g desplazada:

```
11010001101000
01101000110100
00110100011010
00011010001101
```

$h(x) = \dfrac{x^{14}+1}{g(x)} = 1 + x + x^2 + x^4 \rightarrow$ `11101`; **invertido** `10111`. H (10×14), cada fila es `10111` corrida un lugar:

```
10111000000000
01011100000000
00101110000000
00010111000000
00001011100000
00000101110000
00000010111000
00000001011100
00000000101110
00000000010111
```

c) ¿$R(x) =$ `11110001101000` **es válida?** $R = 1 + x + x^2 + x^3 + x^7 + x^8 + x^{10}$. $R + g = x^2 \rightarrow$ **resto** $x^2 \neq 0 \rightarrow$ **no es válida**.

d) Corrección. El síndrome x^2 coincide con el de un error simple en la posición 2 ($2 < n - k = 10$). Corregida: $R + x^2 =$ `11010001101000` $= g(x) \rightarrow$ resto 0 ✓ (la palabra del código para $i(x) = 1$).

e) Entrelazado de 25 códigos. $t = 2$, $L = 25$: ráfaga corregible $= L \cdot t =$ **50 bits**.

2) ISBN-13 `978-1-80056-662-3`**.** Pesos 1,3,1,3,…:
$9 + 21 + 8 + 3 + 8 + 0 + 0 + 15 + 6 + 18 + 6 + 6 + 3 = 103 \rightarrow 103 \bmod 10 = 3 \neq 0 \rightarrow$ **no es válido**. Si se supone que el error está en el dígito de control: los 12 primeros suman $100 \rightarrow$ el control correcto es $(10 - 0) \bmod 10 = 0 \rightarrow$ `978-1-80056-662-0` es válido. (Solo se puede «corregir» bajo ese supuesto: el código detecta el error pero no sabe en qué dígito está.)

3) Código producto. Filas Hamming ($d_1 = 3$), columnas paridad simple ($d_2 = 2$): $d_{min} = d_1 \cdot d_2 = 6 \rightarrow$ corrige 2, detecta 5. La palabra del enunciado

```
1001
0001
1111
000
```

no es válida: en la fila 2 (`0,0,0`) la suma es 0 pero su paridad es 1, y en la columna 2 (`0,0,1`) la suma es 1 pero su paridad es 0 → fila 2 y columna 2 fallan: se cruzan en el bit (fila 2, columna 2). **Cómo se corrige:** la fila indica la posición dentro de la fila (Hamming) y las columnas con paridad errónea indican *cuál columna* tiene el error.

4) Afín $(a, b) = (13, 9)$**,** $n = 40$, alfabeto `ABCDEFGHIJKLMNÑOPQRSTUVWXYZ0123456789 /,` (40 símbolos), criptograma `U5K5CRUPKNLWNNWZAZY9K8EFQDURKJ886ERUPKIUKÑQ7`.

- $40 = 2^3 \cdot 5 \rightarrow \varphi(40) = 40 \cdot \frac{1}{2} \cdot \frac{4}{5} = 16$.
- $a^{-1} = 13^{\varphi(40)-1} = 13^{15} \bmod 40 =$ **37** (comprobado: $13 \cdot 37 = 481 = 12 \cdot 40 + 1$).
- $M_i = (C_i - 9) \cdot 37 \bmod 40$.
- Texto: `EL LUNES 17/11/2025, COMPREN ACCIONES DE YPF` (verificado).
- Si $n = 1\,000\,003$ (primo) y $a = 139$: $\varphi(n) = 1\,000\,002$, $a^{-1} = 139^{1\,000\,001} \bmod 1\,000\,003 =$ **107 914**.

5) ¿Qué algoritmo mantiene H_0 **pero altera las de orden superior?** La **transposición** (ver 2.5 y 12): conserva las frecuencias de cada símbolo (H_0 igual) y destruye las dependencias entre símbolos vecinos ($H_1, H_2, \ldots$ cambian).

6) Filtro paso bajo: el de **mediana** (o el de media/gaussiano); reduce ruido a costa de emborronar bordes. Sobre la ventana de 3.1: mediana 14.

5.2 Parcial 2020, 15-12 (código (12,3) con $h(x) = x^3 + x^2 + x + 1$, ISBN-10, afín $n = 29$, imágenes)

1) Control de error.

- **A)** $k = \deg h = 3$ (el código es (12,3)).
- **B)** $g(x) = \dfrac{x^{12}+1}{h(x)} = x^9 + x^8 + x^5 + x^4 + x + 1$ (`1100110011`). Pesos de las palabras: 6 y 12 → $d_{min} = 6$: como **detector** detecta hasta 5; como **corrector** corrige 2 (y detecta un tercero).
- **C)** $g(x) = 1 + x + x^4 + x^5 + x^8 + x^9$.
- **D)** Matrices. G no sistemática (3×12): filas `110011001100` , `011001100110` , `001100110011` . En forma sistemática $G = [P \mid I_3]$ con P filas `110011001` , `101010101` , `100110011` ; $H = [I_9 \mid P^T]$ (9×12). (La resolución de alumna usó esta misma forma; su G coincide con el calculado, el H manuscrito no lo revisé fila por fila.)
- **E) Triple error y síndrome.** Al dividir el mensaje erróneo por $g(x)$ el resto no es cero → **se detecta** el error. Con 3 errores el síndrome es la **suma (XOR) de 3 columnas de** H; como $d_{min} = 6$, ese síndrome **no puede coincidir con una columna ni con la suma de 2** (si coincidiera, la suma de ambos patrones sería una palabra válida de peso ≤5, menor que 6). Por eso el triple error **siempre se detecta** (el código detecta hasta 5) pero **no se corrige**: solo se lo puede «explicar» con 3 columnas, más que la capacidad $t = 2$. Peor: hay síndromes (se contaron 55 en este código) que se explican con **dos combinaciones distintas de 3 columnas** (el mensaje está a distancia 3 de dos palabras válidas), y el decodificador no sabría a cuál corregir; si lo obligaran a corregir, podría devolver una palabra equivocada.

Aparte (2010): decodificar `110011001110` con el mismo código (mayores potencias a la derecha): resto $x^8 + x^6 + x^4 + x^2 + 1 \neq 0$ → error; coincide con el síndrome de un error simple en la posición x^{10} → corregida `110011001100` (resto 0).

2) ISBN-10 `3-540-?3218-9` **.**

- **A)** Pesos 10,9,…,1: $3 \cdot 10 + 5 \cdot 9 + 4 \cdot 8 + 0 \cdot 7 + x \cdot 6 + 3 \cdot 5 + 2 \cdot 4 + 1 \cdot 3 + 8 \cdot 2 + 9 \cdot 1 = 158 + 6x$. Debe ser múltiplo de 11: $158 \bmod 11 = 4$, $6x \equiv 7 \pmod{11} \rightarrow x = 7 \cdot 6^{-1} = 7 \cdot 2 = 14 \equiv$ **3**. Comprobación: $158 + 18 = 176 = 16 \cdot 11$ ✓ → `3-540-33218-9` . *(La resolución de alumna dio 6 por haber sumado 113 en vez de 158; ver sección 12.)*
- **B)** Es un **código pesado** (cada dígito se pondera por su posición); $d_{min} = 2$: detecta 1 error, no corrige.
- **C)** Base prima: ver 1.4 (inverso único, decodificación unívoca).

3) Afín $(a, b) = (3, 11)$**,** $n = 29$, alfabeto `A=0,…,Ñ=14,…,Z=26,$=27,#=28` , criptograma `SGHPKQ$SGT` .

- $a^{-1} = 3^{-1} \bmod 29 = 10$ ($3 \cdot 10 = 30 \equiv 1$).
- $M = 10 \cdot (C - 11) \bmod 29$ (equivalente a $a^{-1}C + b'$ con $b' = -10 \cdot 11 \bmod 29 = 6$).
- Descifrado: `VIRUS COVID` (el `$` es el espacio).

4) Imágenes: vectorial vs. bitmap y RLE de PCX (sección 3).

5.3 Parcial 2010: Vigenère + permutación y entropías

Criptograma `LFXOJKLWEOPKYDJQPJUCADCG` ; Vigenère `RACING` y luego permutación $(6, 5, 4, 3, 2, 1)$, alfabeto de 26 letras con $A = 1 \ldots Z = 26$.

- **A)** Deshacer la permutación (invertir cada bloque de 6) → `KJOXFLKPOEWLJPQJDYGCDACU` ; deshacer Vigenère con $A = 1$ → `SILORESOLVIERONAPROBARON` (*«si lo resolvieron aprobaron»*).
- **B)** H_0: original **3,268** bits; tras Vigenère **3,887**; tras permutación **3,887**.
- **C)** La permutación **no** modifica H_0 (sí las de orden ≥1); Vigenère **modifica** H_0 y todas.
- **D)** Redundancia de la fuente original: $1 - H/H_{max} = 1 - 3{,}268/\log_2 26 = \mathbf{30{,}5\%}$.
- **Teóricas:** ordenar por entropía `.rar` > `.exe` > `.txt` ; entrelazar 7 códigos del (12,3): ráfaga de $7 \cdot 2 = 14$ bits; conviene comprimir la **original**.

5.4 Parciales 2012 (T1 y T2): ¡el trabajo es el mismo!

- **T1, (15,K)** con $h = x^4 + x + 1$: $k = 4$, $g(x) = x^{11} + x^8 + x^7 + x^5 + x^3 + x^2 + x + 1$, $d_{min} = 8$ (las 15 palabras no nulas pesan 8): corrige 3, detecta 7. $R(x) = x^8 + x^4 + x^2$: resto $= x^8 + x^4 + x^2 \neq 0$ → no es válida; el único patrón de peso ≤3 con ese síndrome es **el mismo** $e(x) = x^2 + x^4 + x^8$ → corregida = **palabra nula** (error triple, justo en la capacidad $t = 3$). Con 20 códigos entrelazados: ráfaga de $20 \cdot 3 = 60$.
- **T2, (14,K)** con $h = x^4 + x^2 + x + 1$: $k = 4$, $g(x) = x^{10} + x^8 + x^7 + x^3 + x + 1$, $d_{min} = 6$. $R(x) = x^8 + x^4$: resto $= x^8 + x^4$; no es error simple pero sí **doble** en las posiciones 4 y 8 → corregida = palabra nula (válida). Ráfaga con 20 entrelazados: $20 \cdot 2 = 40$.
- **Cripto T1:** transposición 3×4 clave $(4, 1, 3, 2)$ + Vigenère `CANCUN` ; criptograma `GAUQ YBXM OÑSK` . Se deshace Vigenère (`EAIOEOVMCMYX`) y la transposición (con la convención «la columna j va a la posición k_j») → `MEVOYAMEXICO` (*«me voy a México»*).

- **Cripto T2:** Vigenère `AZULES` + matriz 4×4 (lectura por columnas en el orden $(2,4,1,3)$); criptograma `BOEU MIHJ FDFX DVDT` → `MEHUEDOENSANJUAN` (*«me huedo en San Juan»* / *«me quedo…»*; así queda, es el resultado que da el algoritmo). *Cuál da más seguridad:* Vigenère (polialfabético, cambia H_0) frente a la transposición (solo reordena).
- **Teóricas del T2:** necesidad de la base prima (1.4); ISBN-10 `234323?323` → dígito **0** (la suma ponderada con el 0 da 143 = 13·11); tabla de Galois/Reed-Solomon (1.11).

5.5 Parcial 2013 T2 (ISBN-10)

`1-122-?3445-5`: $10+9+16+14+6x+15+16+12+10+5 = 107+6x$; $107 \bmod 11 = 8$; $6x \equiv 3 \pmod{11} \rightarrow x = 3\cdot 2 = 6$ → **6** ($107+36 = 143 = 13\cdot 11$ ✓). Capacidad: $d_{min} = 2$ (detecta 1, no corrige).

(Los criptogramas de 2013 T1 y T2 —Vigenère `KERBEROS` / `HELLMAN` + transposición 4×5/3×7— no me dieron texto con sentido probando las convenciones de clave, orden y alfabeto de 27 letras; probablemente el enunciado tiene alguna errata. El método es el de 4.7; si lo resolvés a mano, verificalo.)

5.6 Práctico de Criptografía 2026 (Área Temática 2)

1–3. César. $k = 4$: `EO TEQ TEQ, C EO ZMQS...`; $k = 18$: `RC HRE HRE, P RC NZEG...`; sí hay estructura de grupo (2.4). D_k de `YAMNMYOAYPY` con $k = 12$ → `NOABANDONEN`. Transposición de bloques $n = 7$, clave $(3,7,2,6,4,1,5)$ de `NO POR MUCHO MADRUGAR…` (se rellena con `X`): `PUOMONRORHDMCAAXGXRUX`; **clave de descifrado** $(6,3,1,5,7,4,2)$ (la permutación inversa), que devuelve `NOPORMUCHOMADRUGARXXX`.

4–5. Vigenère con clave de largo 8: $K_i = C_i - M_i$ → `COLOSSUS` (se repite y verifica en los 32 caracteres).

6. Transposición de transposición: ver 2.4. **7.** `MXDQLWR` → `JUANITO` ($k = 3$).

8. Variante de César «por bloques» $E_K(m) = am + b \bmod N$:

- **a)** Con N primo: $a \in \{1,\dots,N-1\}$ y $b \in \{0,\dots,N-1\}$ → $\mathbf{N(N-1)}$ claves ($N = 131$: 17 030).
- **b)** $N = 131$, $a = 3$, $b = 7$, un carácter por bloque (ASCII, sin espacios): `HAY GENTE QUE…` → 92, 71, 12, 89, 83, 110, 128, 83, 119, 0, 83, 80, 83, 125, 0, 77, 83, 110, 77, 95, 71, 128, 95, 83, 110, 83, 104, 71, 77, 71, 74, 83, 15, 71, 104, 104, 83, 110, 71.
- **c)** $D_k(c) = a^{-1}(c-b) = a^{-1}c + d$ con $a^{-1} = 44$ ($3\cdot 44 = 132 \equiv 1$) y $d = -44\cdot 7 \bmod 131 = 85$: $D_k(c) = 44c + 85 \bmod 131$ (comprobación: $D(92) = 72 =$ `H`).
- **d)** Se prueba el par (a,b) de modo que los símbolos más frecuentes del criptograma (100, 99, 109, 60…) correspondan a `e, a, o, l, s`: da la clave de cifrado $a = 10$, $b = 7$ (para descifrar: $M = 118\,(C-7) \bmod 131$, porque $10^{-1} \bmod 131 = 118$) y el mensaje `Es mejor que aprender mucho el aprender cosas buenas.`

9. LFSR $f(x) = x^5 + x^2 + 1$**, semilla** `10000`**:** período **31**; secuencia `00001010111011000111111001101001`; `ENIGMA` → ver 2.7.

10. RSA $p = 97$**,** $q = 31$**:** $n = 3007$, $\varphi = 96\cdot 30 = 2880 = 2^6\cdot 3^2\cdot 5$. **Válidas**: las e coprimas con 2880. $e = 24$ ✗ (par), $e = 33$ ✗ (múltiplo de 3), $e = 45$ ✗ (múltiplo de 3 y 5), $e = 49$ ✓ (coprimo; $d = 529$). **Solo la d).**

11–12. Paco y Lola (alfabeto de 28: `A=0…Ñ=14…Z=26`, blanco=27). Paco: $n = 35$, $\varphi = 24$, $e = 11$, $d = 11$. Lola: $n = 33$, $\varphi = 20$, $e = 7$, $d = 3$.

- **Paco→Lola** `SI` (S=19, I=8), cifrando con la pública de Lola ($n = 33$, $e = 7$): $19^7 = 13$, $8^7 = 2$ → $M_{Paco} = (13,2)$. **Firma** de la `S` con la privada de Paco: $19^{11} \bmod 35 = 24$ (verificación: $24^{11} \bmod 35 = 19$ ✓).
- **Lola→Paco** `NO` (N=13, O=15), con la pública de Paco ($n = 35$, $e = 11$): $13^{11} = 27$, $15^{11} = 15$ → $M_{Lola} = (27,15)$ (el `O` es no cifrable: $15^{11} \equiv 15 \pmod{35}$). **Firma** de la `N` con la privada de Lola: $13^3 \bmod 33 = 19$ (verificación: $19^7 \bmod 33 = 13$ ✓).
- **Descifrar y verificar:** Lola: $13^3 = 19$, $2^3 = 8$ → `SI`; verifica la firma con la pública de Paco. Paco: $27^{11} \bmod 35 = 13$, $15^{11} = 15$ → `NO`.
- *Observación:* hay que cuidar que **el valor firmado quepa** en el módulo del que lo va a procesar (acá 24 < 33 y 19 < 35; si no, hay que firmar y cifrar por separado).

13. $(n,e) = (2291, 17)$**,** $C = 575$**.** Fermat: $2291 = 79\cdot 29$; $\varphi = 78\cdot 28 = 2184$; $d = 257$; $M = 575^{257} \bmod 2291 = 65$ → **ASCII 65 =** `A`.

14. $(n,e) = (536\,813\,567,\ 3\,602\,561)$**.** Fermat (13 695 iteraciones) da $n = 65\,537\cdot 8191$ (¡un primo de Fermat y un primo de Mersenne!); $\varphi = 65\,536\cdot 8190 = 536\,739\,840$; $\gcd(e,\varphi) = 1$; $d = 201\,934\,721$.

15. Investigación: *exponenciación rápida* (4.9) y *factorización*: división por tentativas, **Fermat** (cuando $p \approx q$), *rho de Pollard*, *criba cuadrática* y *criba general del cuerpo de números* (la más rápida conocida).

5.7 Ejemplos de las presentaciones, ya verificados

Tema	Ejemplo	Resultado
RSA	$n = 91$, $e = 5$, $d = 29$, $M = 48$	$C = 55$, $55^{29} \bmod 91 = 48$
Diffie-Hellman	$p = 1999$, $\alpha = 33$, $a = 47$, $b = 117$	secreto 1506
Congruencial	$a = 5, b = 1, n = 16, x_0 = 10$	10, 3, 0, 1, 6, 15, 12, 13, 2, 11, 8, 9, 14, 7, 4, 5

Tema	Ejemplo	Resultado
LFSR	$x^4 + x + 1$, `1001`	`100100011110101`
Mensajes no cifrables	$n = 35$, $e = 11$	{0,1,6,14,15,20,21,29,34}
RS(7,3,5) sobre $GF(8)$	$g(x)$	$x^4 + \alpha^3 x^3 + x^2 + \alpha x + \alpha^3$
Hamming (7,4)	$g = x^3 + x + 1$, info $x^2 + 1$	no sist. $x^5 + x^2 + x + 1$; sist. $x^5 + x^3 + x^2$

---

6. Chuleta de fórmulas

Control de error

Concepto	Fórmula
Redundancia	$\dfrac{n-k}{n}$
Distancia mínima (lineal)	peso mínimo de una palabra no nula
Detecta / corrige	$d_{min} - 1$ / $t = \left\lfloor \dfrac{d_{min} - 1}{2} \right\rfloor$; $d_{min} \geq 2t + 1$
Corrector + detector	corrige t, detecta $d_{min} - 1 - t$
Singleton	$d_{min} \leq n - k + 1$
Cota de Hamming	$2^{n-k} \geq \sum_{i=0}^{t} \binom{n}{i}$
Producto	$d_{min} = d_1 \cdot d_2$
Generador cíclico	$g(x) \mid (x^n + 1)$, $\deg g = n - k$; $h(x) = (x^n + 1)/g(x)$
Codificación sistemática	$V = x^{n-k} i(x) + \left(x^{n-k} i(x) \bmod g \right)$
Síndrome	$s = R \bmod g$; $s = r\,H^T$
G, H	$G = [I \mid P]$, $H = [P^T \mid I]$; $G\,H^T = 0$
Entrelazado	ráfaga $= L \cdot t$
RS	$n = 2^m - 1$, $n - k = 2t$, $d_{min} = 2t + 1$, $g = \prod_{i=1}^{2t} (x - \alpha^i)$
ISBN-10	$\sum (11 - i) x_i \equiv 0 \pmod{11}$
ISBN-13	pesos 1,3,… $\equiv 0 \pmod{10}$
Base 37	$S + P \equiv 0 \pmod{37}$

Criptografía

Concepto	Fórmula
César	$C = (M + k) \bmod n$
Afín	$C = (aM + b) \bmod n$; $M = (C - b)a^{-1}$; $\gcd(a, n) = 1$; $\#\text{claves} = \varphi(n)\,n$
Vigenère	$C_i = (M_i + K_{i \bmod L}) \bmod n$
Hill	$C = KM \bmod n$, $\gcd(\det K, n) = 1$
Inverso	$a^{-1} = a^{\varphi(n)-1} \bmod n$
Euler	$\varphi(n) = n \prod (1 - 1/p)$; $\varphi(p) = p - 1$; $\varphi(pq) = (p-1)(q-1)$
Shannon perfecto	$I(M; C) = 0$; $\lvert K \rvert \geq \lvert M \rvert$, equiprobable, un solo uso
LFSR	$T = 2^n - 1$ si f primitivo; Berlekamp-Massey: $2n$ bits
DH	$K = \alpha^{ab} \bmod p$
RSA	$n = pq$; $\varphi = (p-1)(q-1)$; $\gcd(e, \varphi) = 1$; $d = e^{-1} \bmod \varphi$; $C = M^e$; $M = C^d$
ElGamal (cifrar)	$r = \alpha^k$, $t = m\beta^k$; $m = t\,r^{-a}$
ElGamal (firma)	$r = \alpha^k$, $s = (H - ar)k^{-1} \bmod (p-1)$; $\beta^r r^s \equiv \alpha^H$
Claves simétricas	$n(n-1)/2$
DES	bloque 64, clave 56, 16 vueltas, 8 S-cajas 6→4, expansión 32→48
AES	bloque 128; claves 128/192/256; 10/12/14 rondas
IDEA	bloque 64, clave 128, 8 vueltas, 52 subclaves de 16 bits
Hash	MD5 128 b; SHA-1 160 b; bloques de 512

7. Preguntas teóricas cerradas (con respuesta breve)

1. **Si se entrelazan n códigos, ¿qué nueva capacidad adquiere el código?** Capacidad de corregir **ráfagas** de hasta $n \cdot t$ errores seguidos (con t la capacidad aleatoria de cada código).
2. **Propiedades de $g(x)$ de un cíclico.** (1) grado $n-k$; (2) factor de x^n+1; (3) coeficientes 1 en el término independiente y en el de mayor grado; (4) grado mínimo; (5) toda $V(x)$ es múltiplo de $g(x)$ y todo múltiplo es palabra válida; (6) toda rotación de una palabra válida es válida.
3. **¿Qué características tienen G y H entre sí?** Son **ortogonales** ($G\,H^T = 0$): $G = [I \mid P] \Rightarrow H = [P^T \mid I]$. Una palabra válida multiplicada por H^T da 0; si hubo error da el síndrome.
4. **Capacidad de un código ISBN.** Detecta **un** error (cualquier posición), no corrige; con ≥2 errores la suma puede coincidir y dar un ISBN válido de otro libro (error indetectable). $d_{min} = 2$.
5. **Un cíclico con distancia mínima 10.** Corrector: corrige 4 y detecta 5; detector: detecta 9.
6. **¿Por qué la base prima?** Unicidad del inverso y decodificación unívoca; no se generan ambigüedades.
7. **Confusión y difusión.** Ver 2.4 (sustitución y transposición).
8. **Niveles de entropía que modifica cada algoritmo.** Ver 2.5.
9. **Simétrica vs. asimétrica; híbrido.** Ver 2.8.
10. $I(A;B)$ **en un buen criptosistema.** Mínima, idealmente 0.
11. **¿Qué fuente conviene comprimir?** La original (antes de cifrar).
12. **Orden por entropía de** `.exe`, `.txt`, `.rar`. `.rar` > `.exe` > `.txt`.
13. **¿Una transposición de una transposición es más segura?** No si tienen el mismo largo de bloque (es otra permutación); mejor con largos distintos.
14. **¿César tiene estructura de grupo?** Sí (abeliano); por eso el cifrado múltiple no ayuda. DES no.
15. **Shannon: condiciones para el secreto perfecto.** Clave tan larga como el mensaje, equiprobable y de un solo uso.
16. **¿Por qué se cifra el hash y no el documento en una firma digital?** Porque el cifrado asimétrico es lento; el hash es corto y de largo fijo y firma la «huella» del documento.
17. **¿Qué es un criptosistema perfecto en términos de entropía?** $H(M) = H(M \mid C)$, es decir $I(M;C) = 0$.
18. **LFSR con polinomio primitivo, ¿qué debilidad?** Con $2n$ bits conocidos se reconstruye toda la secuencia (Berlekamp-Massey).
19. **¿Por qué ElGamal produce un criptograma del doble de largo?** Porque cifra cada bloque como un **par** (r, t).
20. **¿Qué es una función unidireccional con trampa?** Fácil de calcular en un sentido, intratable en el inverso salvo que se conozca la «trampa» (factorización, logaritmo discreto).

---

8. Cómo está redactado el parcial: trampas del enunciado

- **Te dan $h(x)$ y a veces lo llaman «generador».** Hay que calcular $g(x) = (x^n+1)/h(x)$.
- **«Mayores potencias a la derecha».** El primer bit es x^0. Si te dan $R(x) =$ `11110001101000`, es $1 + x + x^2 + x^3 + x^7 + x^8 + x^{10}$.
- **«Capacidad» pide las tres cosas:** como detector, como corrector, y combinadas (corrige t y detecta $d - 1 - t$). Y suele agregarse **redundancia** y **ráfaga**.
- **El criptograma compuesto se descifra al revés.**
- **Alfabeto:** el parcial dice cuál (26, 27, 29, 40); cambia n y los inversos. Con $A = 1$ en lugar de $A = 0$ hay un corrimiento de 1.
- **Clave de la transposición:** dos convenciones; la que da texto con sentido es la correcta.
- **«¿Es posible corregirlo?» (ISBN-13):** solo si se supone en qué dígito está el error.
- **«¿Qué algoritmo mantiene H_0 pero cambia las demás?»** Transposición.
- **Las firmas se calculan con la clave privada del emisor; el cifrado, con la pública del receptor.**

---

9. Mapa conceptual del Parcial 2

```
PARCIAL 2
├─ CONTROL DE ERROR
│   ├─ distancia de Hamming → dmin → capacidad (detecta d−1, corrige ⌊(d−1)/2⌋)
│   ├─ paridad / códigos pesados (base 37, ISBN-10 mod 11, ISBN-13 mod 10)
│   ├─ arreglos: rectangular, triangular, cúbico, producto (d = d1·d2)
│   ├─ Hamming (7,4) perfecto, síndrome = posición; extendido (8,4)
│   ├─ lineales: G=[I|P], H=[Pᵀ|I], síndrome = suma de columnas de H
│   ├─ cíclicos: g(x) | xⁿ+1, h=(xⁿ+1)/g; sistemático; resto = síndrome; LFSR
│   ├─ BCH / Reed-Solomon (GF(2^m), n=2^m−1, d=2t+1, MDS)
│   └─ entrelazado (ráfaga = L·t), convolucionales, turbo (cerca de Shannon)
├─ CRIPTOGRAFÍA
```

```
│   ├─ pilares, criptosistema, Shannon perfecto (I(M;C)=0)
│   ├─ simétricos clásicos: sustitución (confusión) / transposición (difusión)
│   │    César, afín, homofónica, Hill, Vigenère (Kasiski, IC), transposiciones
│   ├─ simétricos modernos: Feistel, DES, 3-DES, IDEA, AES; modos ECB/CBC
│   ├─ flujo: XOR + secuencia cifrante; congruencial, LFSR primitivo, NLFSR
│   └─ asimétricos: DH, RSA, ElGamal, ECC; hash (MD5/SHA); firma; híbrido
└─ IMÁGENES: puntuales, filtros (media, mediana, gauss, paso alto), máscaras, histograma, RLE
```

---

10. Dónde está cada cosa (referencias cruzadas)

Pregunta típica	Sección
¿Cuánto vale d_{min} de mi código?	0.2, 4.1
¿Cómo armo G y H?	1.9, 4.2
¿Cómo corrijo $R(x)$?	4.4, 5.1
Dígito faltante del ISBN	1.4, 5.2, 5.5
Descifrar un compuesto	4.7, 5.3, 5.4
Entropías y criptografía	2.5, 5.3
RSA, firmas y Paco-Lola	2.9, 4.9, 5.6
LFSR	2.7, 4.10
Tabla de Reed-Solomon	1.11
Híbrido, simétrico vs. asimétrico	2.8, 7

---

11. Cosas del material que conviene saber

- Las **diapositivas y los apuntes** usan notaciones distintas para lo mismo (polinomios con mayor potencia a la izquierda o a la derecha; LFSR de Fibonacci vs. Galois). Elegí una y declarala.
- El apunte de Fabricio tiene secciones **marcadas por completar** (decodificación de Reed-Solomon, ejemplo de LFSR para Hamming) y una nota propia: «LFSR de Fibonacci en cripto, de Galois en control de error».
- El **material se repite**: los mismos tres códigos y las mismas teóricas. Resolvé primero los de 0.2 y las preguntas de la sección 7.

---

12. Errores que aparecen en las resoluciones de alumnos (y cómo evitarlos)

Se revisó la resolución del **parcial 17-11-2025 T1** y la del **parcial 15-12-2020** de las carpetas. **Todo lo que sigue está verificado por cálculo.**

#	Resolución	Qué dice	Qué es lo correcto
1	2025, 1b) matriz H	arma H con $h(x) =$ `11101` **sin invertir**	H usa $h(x)$ **invertido** (`10111`); sin invertir **no es ortogonal a** G
2	2025, 1a) tabla de pesos	pesos 6, 7, 8 y 9	los pesos reales son **6, 8 y 14** (una sola palabra de peso 14). $d_{min} = 6$ sí está bien
3	2025, 1d)	escribe « `R corregida = 1110001101000` » (13 bits)	es `11010001101000` (14 bits, $= g(x)$)
4	2025, 1e)	«$\lfloor B/25 \rfloor = 2$, $25 \cdot 2 = 50$»	resultado correcto (50 bits), pero conviene decir directo $L \cdot t = 25 \cdot 2$
5	2025, ejercicio 5	contesta «sustitución monoalfabética»	la que conserva H_0 y **altera las de orden ≥1** es la **transposición** (la sustitución es una relabelación biyectiva y conserva todos los órdenes)
6	2025, ISBN-13	«es posible corregirlo» sin condición	solo bajo el **supuesto** de que el dígito mal es el de control
7	2020, ISBN-10 `3-540-?3218-9`	sumó 113 y obtuvo **6**	la suma sin el `?` es **158**, el dígito es **3** (`3-540-33218-9` , $176 = 16 \cdot 11$)
8	2020, 1D)	su G en forma sistemática	coincide con el cálculo: $G = [P \mid I_3]$ (el H manuscrito no lo verifiqué fila por fila)

Además, para **no perder puntos**: (a) escribir la convención de los bits; (b) justificar con el síndrome o con la división, no solo dar el resultado; (c) al descifrar un compuesto, **deshacer en orden inverso**; (d) en RSA, verificar $\gcd(e, \varphi) = 1$ y **comprobar** descifrando.

Nota de cobertura

- **Leído y usado:** apuntes de Fabricio (Unidades 7, 8, 9 y 10, y las tablas de símbolos), presentaciones Tema 1 a 5 (el texto de los PPT; las **figuras y esquemas** de las diapositivas —diagramas del DES, tablas IP/PC, esquemas de IDEA— no se extraen como texto, salvo las cajas S y los valores que figuran como texto), Práctico de Criptografía 2026, los 7 parciales y las preguntas teóricas, y la resolución manuscrita de 2020 y la tipeada de 2025.
- **Fórmulas de los PDF:** el apunte de Fabricio perdió algunos símbolos matemáticos al extraer el texto (subíndices, φ, sumatorias); las fórmulas que se usaron se **reconstruyeron y recalcularon**; en los pocos casos dudosos (redundancia de los códigos triangular y cúbico, fórmula del número de polinomios primitivos) está aclarado en el texto.
- **No resuelto:** los criptogramas de los parciales de **2013 T1 y T2** no dieron un texto legible con ninguna de las convenciones probadas; el método es el de 4.7. Hay ejercicios completos con resultado verificado en las secciones 5.1 a 5.6.
- **Complementos míos (marcados):** pasos de ronda de AES, detalles de PCX, ángulos de las máscaras, relación $L \cdot t$ para detección de ráfagas. Verificalos con la cátedra antes de apoyarte solo en ellos.
- **Material personal:** una de las páginas del archivo de la resolución de 2020 es una fotografía de un documento de identidad personal; **no se transcribió** ni se usó.

13. Anexo (págs. 23 a 27): inverso modular con Euler (método del profesor) y con tabla de Euclides extendido; ejercicio 8 del Práctico y casos de parcial.

13. Anexo: inverso modular con Euler (método del profesor) y con Euclides extendido (tabla)

Este anexo amplía 4.6 (descifrar César / afín) y 4.9 (RSA a mano). Todas las cuentas fueron recalculadas por computadora. Hay **dos métodos** para hallar a^{-1} mod N; el método que muestra el profesor en el pizarrón es el **método 1 (Euler + cuadrados sucesivos)**, y el **método 2 (tabla de Euclides extendido)** sirve siempre y se usa para resolver el sistema de ecuaciones de 8.d. Lo ideal: resolver con uno y **comprobar con el otro**.

13.1 ¿En qué tipos de ejercicio se usa?

Tipo de ejercicio	Qué hay que calcular	Método recomendado
Descifrar un afín / «César por bloques» (`E(m) = a·m + b mod N`); Práctico ej. 8.c, parcial 17-11-2025 (afín con `N = 40`)	`a⁻¹ mod N` y la constante `d = −a⁻¹·b mod N`	Euler si `N` primo; Euclides con tabla si `N` compuesto
Hallar la clave `(a, b)` de un afín por frecuencias (Práctico ej. 8.d)	Sistema de 2 ecuaciones; invertir el coeficiente (`55⁻¹ mod 131`)	Tabla de Euclides
RSA a mano (4.9): clave privada	`d = e⁻¹ mod ϕ(n)` (el módulo es `ϕ(n)`, compuesto)	Tabla de Euclides (Euler pediría `ϕ(ϕ(n))`)
Código pesado ISBN-10 con un dígito faltante (1.4)	Resolver `w·x ≡ r (mod 11)`: inverso de `w` módulo 11 (primo)	Cualquiera (números chicos)
Cifrado de Hill (2.4)	Inverso de la matriz: necesita `det⁻¹ mod n`, con `gcd(det, n) = 1`	Tabla de Euclides
Preguntas teóricas	¿Cuándo existe el inverso? ¿Qué es φ(N)/CRR? ¿Cuántas claves hay?	Ver 13.8

13.2 Paso 0 (siempre): ¿existe el inverso?

Existe a^{-1} mod N si y solo si `gcd(a, N) = 1`. Si no se cumple, **no existe y no hay que perder tiempo buscándolo**: se responde «no tiene inverso» y se justifica. El `gcd` se obtiene con Euclides (las divisiones hacia adelante); la tabla del método 2 lo da gratis.

Por qué no puede existir: si `g = gcd(a, N) > 1`, entonces `a·x mod N` es siempre múltiplo de `g` y jamás vale 1. Ejemplo `a = 4, N = 14` (`gcd = 2`): `4·x mod 14` para `x = 0…6` da `0, 4, 8, 12, 2, 6, 10`, todos pares. En el cifrado, `E(m) = 4·m + b` daría el mismo `c` para `m = 0` y `m = 7` (`4·7 = 28 ≡ 0`): **la función no es inyectiva y no se puede descifrar de forma única**.

q	r	s	t	Control: r = s·14 + t·4
—	14	1	0	1·14 + (0)·4 = 14
—	4	0	1	0·14 + (1)·4 = 4
3	2	1	-3	1·14 + (-3)·4 = 2
2	0	-2	7	-2·14 + (7)·4 = 0

La tabla termina en `r = 0` sin pasar por 1, y el último resto distinto de cero es `2 = gcd(4, 14)` ⇒ **no hay inverso**.

Si el enunciado da un N compuesto o un a no coprimo, hay que verificarlo explícitamente (es una trampa de parcial). Con el alfabeto de 27 letras, `a` no puede ser múltiplo de 3 (`gcd(3, 27) = 3`); con `N = 40`, no puede ser par ni múltiplo de 5.

13.3 Método 1: Euler y cuadrados sucesivos (el del profesor)

CRR y φ(N)

- **CRR(N)** = *Conjunto Reducido de Residuos*: los restos módulo `N` coprimos con `N`, `CRR(N) = { r_i / gcd(r_i, N) = 1 }`.
- **φ(N) = #CRR(N)** (función de Euler).
- Ejemplo del pizarrón: `N = 14 = 2·7`. De los restos `1…13` se descartan los pares y el `7` ⇒ `CRR(14) = {1, 3, 5, 9, 11, 13}`, `ϕ(14) = 6`.
- N **primo**: todos los restos `1…N−1` son coprimos ⇒ **`ϕ(N) = N − 1`**. **Solo vale para primos.**

Teorema de Euler y fórmula del inverso

Si `gcd(a, N) = 1`, entonces $a^{\phi(N)} \bmod N = 1$. (Multiplicar todo el CRR por `a` solo lo reordena: con `N = 14, a = 3`, `3·{1,3,5,9,11,13} mod 14 = {3,9,1,13,5,11}`; y $3^6 = 729 = 52 \cdot 14 + 1$.) Como $a \cdot a^{\phi(N)-1} = a^{\phi(N)} \equiv 1$:

```
a⁻¹ = a^(ϕ(N) − 1) mod N          (el recuadro «a⁻¹» con el arco hacia ϕ(N) del pizarrón)
```

Cuadrados sucesivos (*square-and-multiply*)

Hay que calcular $a^k \bmod N$ sin construir el número gigante: se pasa el exponente a potencias de 2, se eleva al cuadrado **reduciendo mod N en cada paso** y se multiplican (también reduciendo) solo las potencias de los bits en 1.

Ejemplo del profesor: $a = 3, N = 131$ (primo ⇒ $\phi = 130$): $3^{-1} = 3^{129} \bmod 131$, y $129 = 128 + 1 = 2^7 + 1$, o sea $3^{129} = 3^{128} \cdot 3$ (es la cadena $((3^2)^2 \bmod 131)^2 \bmod 131)^2 \ldots$ del pizarrón: 9, 81, 11, 12(1)…).

Potencia	Cálculo	Resultado mod 131
3^1	—	3
3^2	$3^2 = 9 = 0 \cdot 131 + 9$	9
3^4	$9^2 = 81 = 0 \cdot 131 + 81$	81
3^8	$81^2 = 6561 = 50 \cdot 131 + 11$	11
3^{16}	$11^2 = 121 = 0 \cdot 131 + 121$	121
3^{32}	$121^2 = 14641 = 111 \cdot 131 + 100$	100
3^{64}	$100^2 = 10000 = 76 \cdot 131 + 44$	44
3^{128}	$44^2 = 1936 = 14 \cdot 131 + 102$	102

$3^{129} = 3^{128} \cdot 3 \equiv 102 \cdot 3 = 306 = 2 \cdot 131 + 44 \quad \Rightarrow \quad 3^{-1} \equiv 44 \pmod{131}$ $\quad (3 \cdot 44 = 132 = 131 + 1$ ✓)

Ejemplo con N compuesto (hay que calcular φ): $a = 3, N = 35$. $\gcd(3, 35) = 1$ ✓; $35 = 5 \cdot 7 \Rightarrow \phi(35) = 4 \cdot 6 = 24$ (con $N - 1 = 34$ estaría mal); $3^{-1} = 3^{23} \bmod 35$, $23 = 16 + 4 + 2 + 1$ (binario 10111):

Potencia	Cálculo	Resultado mod 35
3^1	—	3
3^2	$3^2 = 9 = 0 \cdot 35 + 9$	9
3^4	$9^2 = 81 = 2 \cdot 35 + 11$	11
3^8	$11^2 = 121 = 3 \cdot 35 + 16$	16
3^{16}	$16^2 = 256 = 7 \cdot 35 + 11$	11

$3^{23} = 3^{16} \cdot 3^4 \cdot 3^2 \cdot 3^1$: $\quad 11 \cdot 11 = 121 \equiv 16$; $\quad 16 \cdot 9 = 144 \equiv 4$; $\quad 4 \cdot 3 = 12 \quad \Rightarrow \quad 3^{-1} \equiv 12 \pmod{35}$ $\quad (3 \cdot 12 = 36 = 35 + 1$ ✓)

13.4 φ(N) cuando N no es primo

Dos caminos: (a) **por definición**, contando el CRR (N chico); (b) **factorizando** N:

Caso de N	Fórmula	Ejemplos (verificados)
primo p	$\phi = p - 1$	$\phi(131) = 130$
potencia p^k	$\phi = p^k - p^{k-1}$	$\phi(9) = 6$; $\phi(27) = 27 - 9 = 18$
producto $p \cdot q$ (primos distintos)	$\phi = (p - 1)(q - 1)$	$\phi(15) = 8$; $\phi(35) = 24$; $\phi(33) = 20$
m, n coprimos	$\phi(m \cdot n) = \phi(m) \cdot \phi(n)$	$\phi(14) = \phi(2) \cdot \phi(7) = 6$
general	$\phi(N) = N \cdot \prod(1 - 1/p)$ sobre los primos $p \mid N$	$\phi(12) = 12 \cdot ½ \cdot ⅔ = 4$; $\phi(40) = 40 \cdot ½ \cdot ⅘ = 16$; $\phi(100) = 40$

CRR a mano: $CRR(12) = \{1, 5, 7, 11\}$ (4); $CRR(15) = \{1, 2, 4, 7, 8, 11, 13, 14\}$ (8); $CRR(40)$ tiene 16 elementos (ver 2.9). **Error típico:** con $N = 15, a = 7$ ($\phi = 8$) el inverso es $7^7 = 7^4 \cdot 7^2 \cdot 7 \equiv 1 \cdot 4 \cdot 7 = 28 \equiv 13$ ($7 \cdot 13 = 91 = 6 \cdot 15 + 1$ ✓). Si se usa $N - 1$ (exponente 13) da 7, y $7 \cdot 7 = 49 \equiv 4 \neq 1$ ✗.

13.5 Método 2: Euclides extendido con tabla

Idea: cada resto de Euclides entre N y a se escribe como $r = s \cdot N + t \cdot a$. Se llevan s y t en la tabla junto con las divisiones (sin sustituir hacia atrás). Cuando $r = 1$, vale $1 = s \cdot N + t \cdot a$; $s \cdot N$ desaparece módulo N, por lo tanto **la t de esa fila es a^{-1}**.

1. Filas fijas de arranque: $(r, s, t) = (N, 1, 0)$ y $(a, 0, 1)$.
2. q = cociente entero (r de dos filas atrás) ÷ (r de la fila anterior).
3. **Misma regla para las tres columnas**: $\text{nuevo} = (\text{dos filas atrás}) - q \cdot (\text{fila anterior})$.
4. Se corta cuando $r = 1$ (hay inverso) o cuando $r = 0$ sin haber pasado por 1 (el último $r \neq 0$ es el $\gcd$; si es > 1, **no hay inverso**).
5. Si t es negativo, se le suma N.

6. **Control** en cada fila: `r = s·N + t·a`. Si una fila no cierra, el error está ahí.

`a = 3, N = 131`: la `t` de la fila con `r = 1` es `44` ⇒ $3^{-1} = 44$ (igual que por Euler).

q	r	s	t	Control: r = s·131 + t·3
—	131	1	0	1·131 + (0)·3 = 131
—	3	0	1	0·131 + (1)·3 = 3
43	2	1	-43	1·131 + (-43)·3 = 2
1	1	-1	44	-1·131 + (44)·3 = 1

`a = 3, N = 35`: `t = 12` ⇒ $3^{-1} = 12$ (igual que por Euler).

q	r	s	t	Control: r = s·35 + t·3
—	35	1	0	1·35 + (0)·3 = 35
—	3	0	1	0·35 + (1)·3 = 3
11	2	1	-11	1·35 + (-11)·3 = 2
1	1	-1	12	-1·35 + (12)·3 = 1

Con `t` negativo, `a = 7, N = 15`: `t = −2` ⇒ `−2 + 15 = 13` (igual que por Euler).

q	r	s	t	Control: r = s·15 + t·7
—	15	1	0	1·15 + (0)·7 = 15
—	7	0	1	0·15 + (1)·7 = 7
2	1	1	-2	1·15 + (-2)·7 = 1

RSA (4.9): `e = 5, ϕ(n) = 72` ⇒ $d = 5^{-1} \bmod 72 = 29$ ($5 \cdot 29 = 145 = 2 \cdot 72 + 1$ ✓):

q	r	s	t	Control: r = s·72 + t·5
—	72	1	0	1·72 + (0)·5 = 72
—	5	0	1	0·72 + (1)·5 = 5
14	2	1	-14	1·72 + (-14)·5 = 2
2	1	-2	29	-2·72 + (29)·5 = 1

Afín con `N = 40` (parcial 17-11-2025): $13^{-1} \bmod 40$: $t = -3 \Rightarrow 37$ (por Euler: $\phi(40) = 16$, $13^{15} \bmod 40 = 37$).

q	r	s	t	Control: r = s·40 + t·13
—	40	1	0	1·40 + (0)·13 = 40
—	13	0	1	0·40 + (1)·13 = 13
3	1	1	-3	1·40 + (-3)·13 = 1

Variante equivalente (sustitución hacia atrás) para `131, 3`: $131 = 43 \cdot 3 + 2$, $3 = 1 \cdot 2 + 1$ ⇒ $1 = 3 - 1 \cdot 2 = 3 - 1 \cdot (131 - 43 \cdot 3) = 44 \cdot 3 - 1 \cdot 131$. Es la misma cuenta; la tabla evita los errores de signo.

13.6 ¿Cuál método conviene?

	Método 1 (Euler + cuadrados)	Método 2 (tabla de Euclides)
Necesita	$\phi(N)$ (fácil si N es primo o se factoriza)	Nada más: solo N y a
Funciona si	$\gcd(a, N) = 1$	Siempre; además **detecta** si $\gcd \neq 1$
Riesgo	Errores de cuenta en muchas reducciones mod N; confundir $\phi(N)$ con $N - 1$	Errores de signo (se controlan con $r = s \cdot N + t \cdot a$)
Mejor para	N primo (como el 131 del ejercicio); es lo que muestra el profe	N compuesto, módulo $\phi(n)$ de RSA, y **resolver el sistema de 8.d**

13.7 Hallar la clave de un afín por frecuencias (Práctico 8.d), paso a paso

Enunciado: $N = 131$, criptograma de 45 números, el último es un signo de puntuación, letras más frecuentes en castellano `e, a, o, l, s`, ASCII, sin espacios. **La clave no se conoce** (hay 2 incógnitas, `a` y `b`) ⇒ hacen falta **dos pares letra → número**, cada uno da una

ecuación `c = a·m + b mod N`.

1. **Frecuencias.** `100` (8 veces), `99` (5), `109` (4), `60` (4), `69, 129, 59` (3 c/u)…
2. **Suposiciones.** `e` (ASCII 101) `→ 100` (el más frecuente). El último número, `74`, es puntuación; el punto (ASCII 46) `→ 74`.
3. **Sistema:** `101·a + b ≡ 100` y `46·a + b ≡ 74 (mod 131)`. Restando se elimina `b`: **`55·a ≡ 26 (mod 131)`**.
4. **Chequear que 55 sea invertible** (`gcd(55, 131) = 1`) **y calcular** 55^{-1} **con la tabla:**

q	r	s	t	Control: r = s·131 + t·55
—	131	1	0	1·131 + (0)·55 = 131
—	55	0	1	0·131 + (1)·55 = 55
2	21	1	-2	1·131 + (-2)·55 = 21
2	13	-2	5	-2·131 + (5)·55 = 13
1	8	3	-7	3·131 + (-7)·55 = 8
1	5	-5	12	-5·131 + (12)·55 = 5
1	3	8	-19	8·131 + (-19)·55 = 3
1	2	-13	31	-13·131 + (31)·55 = 2
1	1	21	-50	21·131 + (-50)·55 = 1

5. `1 = 21·131 + (−50)·55` ⇒ $55^{-1} \equiv -50 \equiv 81$ (`55·81 = 4455 = 34·131 + 1` ✓).
6. **Despejar** `a`**:** `a ≡ 26·81 = 2106 = 16·131 + 10 ⇒ a = 10`.
7. **Despejar** `b`**:** `b ≡ 100 − 101·10 = −910 ≡ −910 + 7·131 = 7 ⇒ b = 7`. Control con la 2.ª ecuación: `46·10 + 7 = 467 = 3·131 + 74` ✓.
8. **Inverso de** `a = 10` **para descifrar** (tabla):

q	r	s	t	Control: r = s·131 + t·10
—	131	1	0	1·131 + (0)·10 = 131
—	10	0	1	0·131 + (1)·10 = 10
13	1	1	-13	1·131 + (-13)·10 = 1

9. $10^{-1} \equiv -13 \equiv 118$ (`10·118 = 1180 = 9·131 + 1` ✓) ⇒ **`D(c) = 118·(c − 7) mod 131 = 118·c + 91`** (`d = −118·7 = −826 ≡ 91`).
10. **Descifrar** cada número (ej.: `c = 42 ⇒ 118·35 = 4130 = 31·131 + 69 ⇒ 69 = E`) y obtener **«Es mejor que aprender mucho el aprender cosas buenas.»**

Cómo se valida la hipótesis. Si el texto sale basura, se prueba otro par con la lista `e, a, o, l, s`. Fijando `e → 100` y probando `a` (ASCII 97) contra otros números: la diferencia de letras es `101 − 97 = 4` ($4^{-1} = 33$) y `a = (100 − x)·33 mod 131`. Con `a → 99` sale `(a, b) = (33, 42)` (basura); con `a → 109`, `(96, 98)` (basura); con `a → 60`, `(10, 7)` ✓ (texto legible). Dos hipótesis independientes que dan la misma clave (`e → 100` con `. → 74`, y `e → 100` con `a → 60`) es la mejor validación.

> **Si** `N` **es compuesto**, antes de dividir hay que calcular `g = gcd(coeficiente, N)`. Si `g = 1`, se invierte como arriba. Si `g > 1`, `c·a ≡ r` tiene solución **solo si** `g` **divide a** `r`, y entonces hay `g` soluciones módulo `N` (se divide todo, módulo incluido, por `g`); después hay que **descartar las que no sean coprimas con** `N`. Ej.: `6·a ≡ 4 (mod 14)`: `g = 2` divide a 4 ⇒ `3·a ≡ 2 (mod 7)` ⇒ `a ≡ 3` ⇒ `a ∈ {3, 10} (mod 14)`; como `gcd(10, 14) = 2`, **solo** `a = 3` **sirve**.

13.8 Preguntas y casos que pueden caer en el parcial

Pregunta / caso	Respuesta corta
¿Cuándo existe $a^{-1} \bmod N$?	Si y solo si $\gcd(a, N) = 1$. Si N es primo, para todo $a \neq 0$.
Me dan un a y un N que no son coprimos. ¿Qué hago?	No existe el inverso: no se busca. Se justifica con el $\gcd$ (la tabla lo da) y se aclara que E no es inyectiva, por lo que no se puede descifrar en forma única.
¿Cuántas claves (a, b) tiene el afín mod N?	N primo: $N \cdot (N - 1)$ (131 ⇒ 17 030). N compuesto: $\phi(N) \cdot N$ (ej. $N = 40$: $16 \cdot 40 = 640$; $N = 27$: $18 \cdot 27 = 486$).
¿Qué son el CRR y φ(N)?	$CRR(N)$ = restos módulo N coprimos con N; $\phi(N) = \#CRR(N)$. Ej.: $CRR(14) = \{1,3,5,9,11,13\}$, $\phi(14) = 6$.
¿Cuánto vale φ(N) si N es primo? ¿Y si no?	Primo: $N - 1$. Compuesto: contar el CRR o factorizar ($(p-1)(q-1)$, $p^k - p^{k-1}$, $N \cdot \prod(1 - 1/p)$). **Nunca $N - 1$.**
¿Por qué $a^{-1} = a^{(\phi(N)-1)} \bmod N$?	Euler: $a^{\phi(N)} \equiv 1$ si $\gcd(a, N) = 1$; entonces $a \cdot a^{(\phi(N)-1)} \equiv 1$.
¿Cómo se calcula $3^{129} \bmod 131$ a mano?	Cuadrados sucesivos reduciendo mod N en cada paso: $129 = 2^7 + 1$ ⇒ $3^{128} \equiv 102$, $3^{129} \equiv 306 \equiv 44$. Si el exponente no es $2^k + 1$: binario y multiplicar solo los bits en 1.
¿Euler o Euclides?	Euler necesita $\phi(N)$; Euclides con tabla siempre sirve y detecta el no coprimo. Para RSA ($d = e^{-1} \bmod \phi(n)$) conviene Euclides.
Obtuve t negativo en la tabla.	Se le suma N hasta caer en $[0, N-1]$ (ej. $-2 + 15 = 13$).
¿Qué es el b^{-1} que escribe el profe en $m_i = a^{-1} \cdot c_i + b^{-1}$?	La constante $d = -a^{-1} \cdot b \bmod N$ (aquí 85), no el inverso multiplicativo de b. Equivale a $m = a^{-1} \cdot (c - b)$.
Me dan dos pares (letra, número) y piden la clave.	Plantear $a \cdot m_1 + b \equiv c_1$, $a \cdot m_2 + b \equiv c_2$; restar ⇒ $(m_1 - m_2) \cdot a \equiv c_1 - c_2$; invertir el coeficiente con tabla (chequeando $\gcd$ con N); $b = c_1 - a \cdot m_1$.
¿Se puede usar $a = 1$ o $a = 0$?	$a = 1$ es el César. $a = 0$ no sirve (todas las letras dan b).

13.9 Esquema para el examen (en este orden)

1. **Chequear** $\gcd(a, N) = 1$ (Euclides hacia adelante). Si no, «no existe el inverso» y se termina.
2. **Calcular** $\phi(N)$: primo ⇒ $N - 1$; compuesto ⇒ CRR o factorización.
3. **Método 1:** $a^{-1} = a^{(\phi(N)-1)} \bmod N$ con exponente en binario y cuadrados sucesivos reduciendo mod N en cada paso.
4. **Método 2 (para comprobar o si no se conoce φ):** tabla q, r, s, t; la t de la fila con $r = 1$ es el inverso (sumar N si es negativa).
5. **Verificar** $a \cdot a^{-1} \equiv 1 \pmod N$.
6. **Función de descifrado:** $D(c) = a^{-1} \cdot c + d$ con $d = -a^{-1} \cdot b \bmod N$, y probar con una letra ($D(E(m)) = m$).
7. **Si hay que hallar la clave:** sistema de dos ecuaciones, restar, invertir el coeficiente con tabla, hallar a, luego b, y validar que el texto resultante tenga sentido.